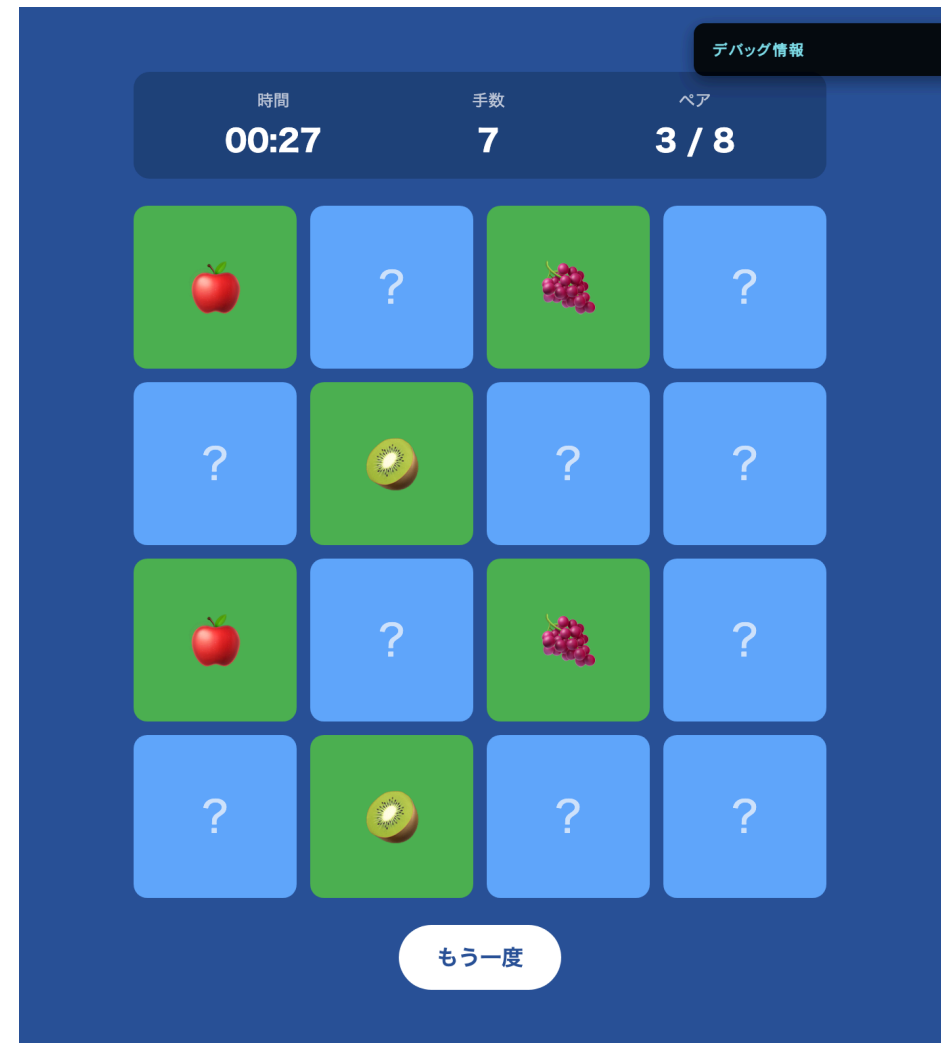


JavaScript で神経衰弱ゲーム開発

今日のゴール

4×4 = 16 枚のカードで遊べる神経衰弱ゲームを作ります。

- カードをクリックするとめくれる
- 2 枚めくって絵柄が一致したらペア成立
- タイマー、手数、ペア数がリアルタイム更新
- 全ペア揃ったら「クリア」表示、「もう一度」ボタンでリセット



今日の流れ

全部で 6 章あります。Chapter 1 のあとに 5 分、Chapter 3 のあとに 10 分の休憩を入れます。

	章	内容
1	JS で画面を組み立てる	カードを盤面に並べる
2	イベントと状態	クリックでカードをめくる
3	状態遷移と非同期	一致判定と待ち時間
4	カードをシャッフル	毎回ちがう並びで遊べるようにする
5	時間で動く画面	タイマー、手数、クリア判定
6	状態を初期化する	リセット機能

前半の 1～3 章でゲームの芯を作り、後半の 4～6 章で遊べる形に仕上げます。

進め方

各章は説明とコードを書く演習の繰り返しです。演習には 2 つのモードがあります。

モード	見分け方	やること
記述	【A】等の穴	構造は見えている中で埋める箇所を考える
コピペ	完成コードが載っている	そのまま手元に貼る (数行の写経も含む)

記述のうち、単語や 1 行で埋まる穴には候補を出します。候補には使わないものも混ざっているので、どれがどこに入るかを考えてください。複数行を書く穴には候補を出さず、代わりに何行になるかを書いています。

同じ記号が 2 回出てくる場合は、どちらにも同じものが入ります。記述のあとは必ず答え合わせのスライドを挟みます。

間に合わなくても大丈夫です。各章のチェックポイントに「追いつき用」のコードを出すので、`script.js` を丸ごと置き換えれば次の章から始められます。

準備: StackBlitz テンプレを開こう

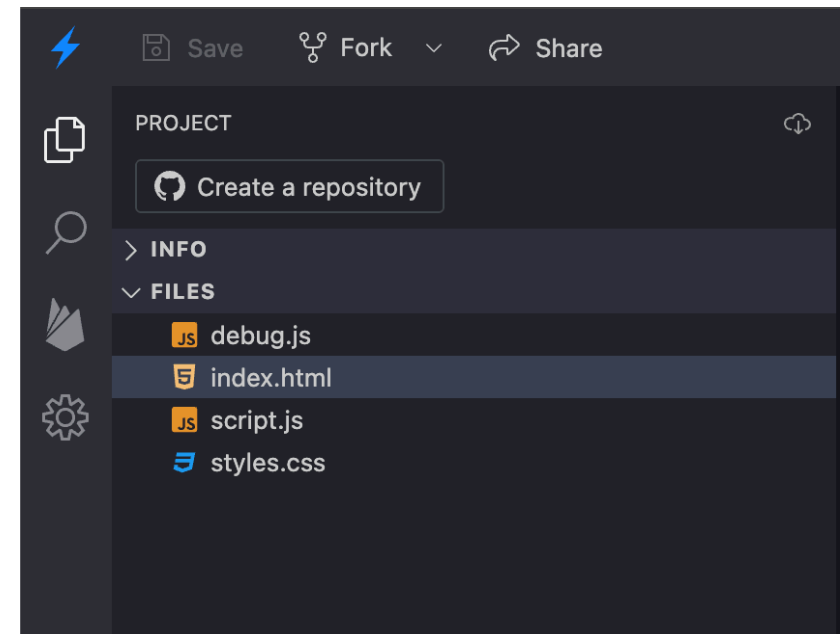
<https://stackblitz.com/edit/web-platform-qnett8m2?file=script.js>

右のようにファイルが 4 つ見えていれば OK です。その右にプレビューが出ます。

プレビューにカードはまだ出ていなくて大丈夫です。

今日さわるのはほぼ `script.js` だけです。最初には `symbols` 配列だけが用意されているので、ここから 1 行ずつ書き足していきます。

新しい関数・変数の追加は、指示がなければ `script.js` のいちばん最後に足します。



準備: 詰まったときは

詰まったら、お気軽に Zoom のチャットなどでメンターにご質問ください。

レビュー右上の黒い小窓は診断パネルです。各演習の **確認** で使います。

デバッグ情報

[コピー](#)

変数など

```
deck          = [Array length=16]
firstCard     = Card(🍎 @0)
secondCard    = null
lockBoard     = false
moves         = (未定義)
matchedPairs  = (未定義)
timerId       = (未定義)
```

盤面 (DOM)

```
カード枚数    = 16
flipped       = 1
matched       = 0
```

診断

問題なし

章ごとのテスト

Chapter 1	10/10 ✓
Chapter 2	4/4 ✓
Chapter 3	0/3
Chapter 4	0/4
Chapter 5	0/9
Chapter 6	0/2

Chapter 1

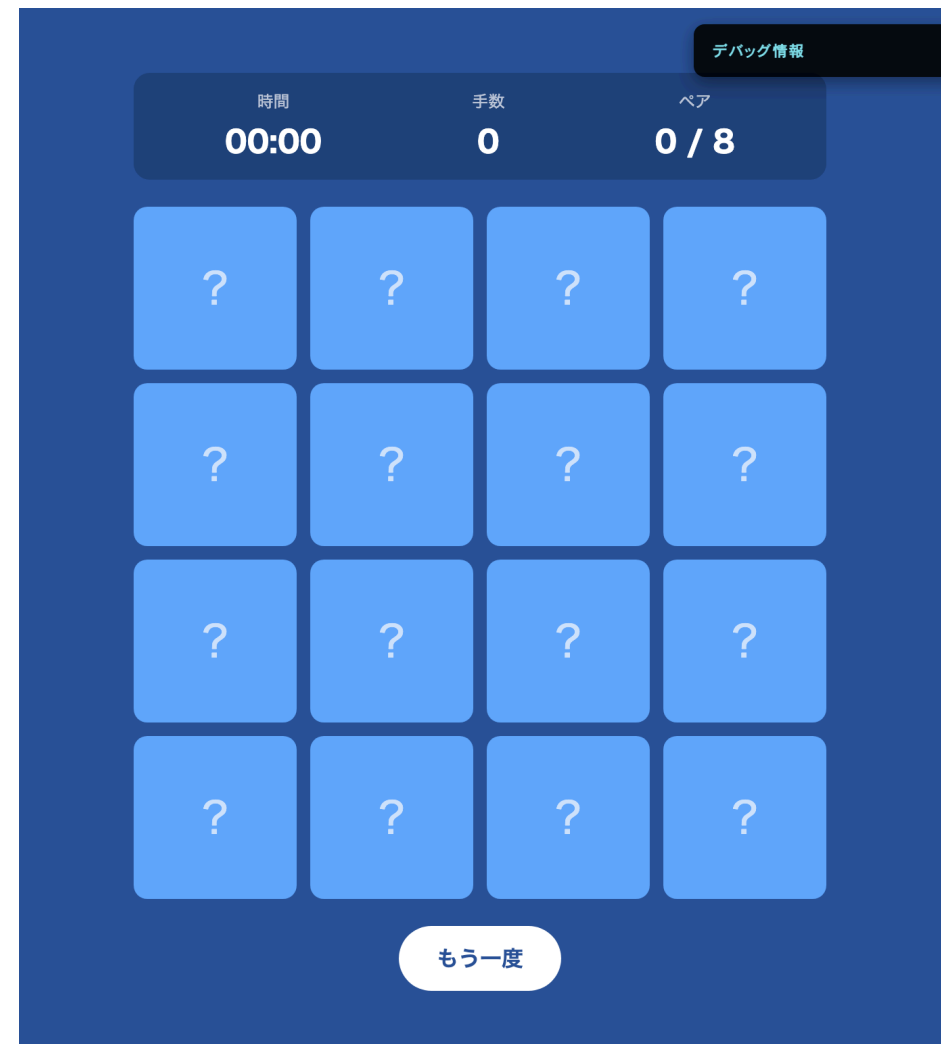
カードを盤面に並べる

Chapter 1 のゴール

16 枚のカードを 4×4 で並べます。

- カードのデータ (絵柄配列) を作る
- カード 1 枚を作る関数を用意する
- 盤面全体を描く関数を作って呼ぶ

この章では JavaScript から HTML/CSS を組み立てていきます。



1-1. カードのデータを作る

`symbols` には 🍏 🍌 🍇 ... と 8 種類の絵柄が入っています。神経衰弱は同じ絵柄のカードが 2 枚ずつあるので、全部で $8 \times 2 = 16$ 枚です。

同じ配列をうしろにもう 1 つつなげれば、どの絵柄もちょうど 2 枚になります。

- `concat` — `A.concat(B)` で A のうしろに B をつなげた新しい配列を返す。A も B も変わらない

例 1: `["a", "b"].concat(["c"])` → `["a", "b", "c"]`

例 2: `["a", "b"].concat(["a", "b"])` → `["a", "b", "a", "b"]`

1-1. deck を作る

コピー script.js のいちばん最後に書き足します。

```
// symbols を 2 回連結して 16 枚の deck を作る  
const deck = symbols.concat(symbols);
```

データを用意しただけで、まだ表示するコードがないので、貼っても画面は変わりません。

確認 診断パネルの「変数など」で `deck` が「(未定義)」から `[Array length=16]` に変わります。

デバッグ情報

コピー ▼

変数など

deck	= [Array length=16]
firstCard	= (未定義)
secondCard	= (未定義)
lockBoard	= (未定義)
moves	= (未定義)
matchedPairs	= (未定義)
timerId	= (未定義)

HTML には空の入れ物だけ置く

16 枚のカードは、`index.html` に `<div class="card">` を 16 個書いても並べられます。ただ手で 16 個書くのは大変で、絵柄を変えるときもカードの枚数を変えるときも、そのぶん書き直しになります。

```
<!-- 16 個書くようになる -->
<div id="board">
  <div class="card">?</div>
  <div class="card">?</div>
  <!-- ...あと 14 個 -->
</div>
```

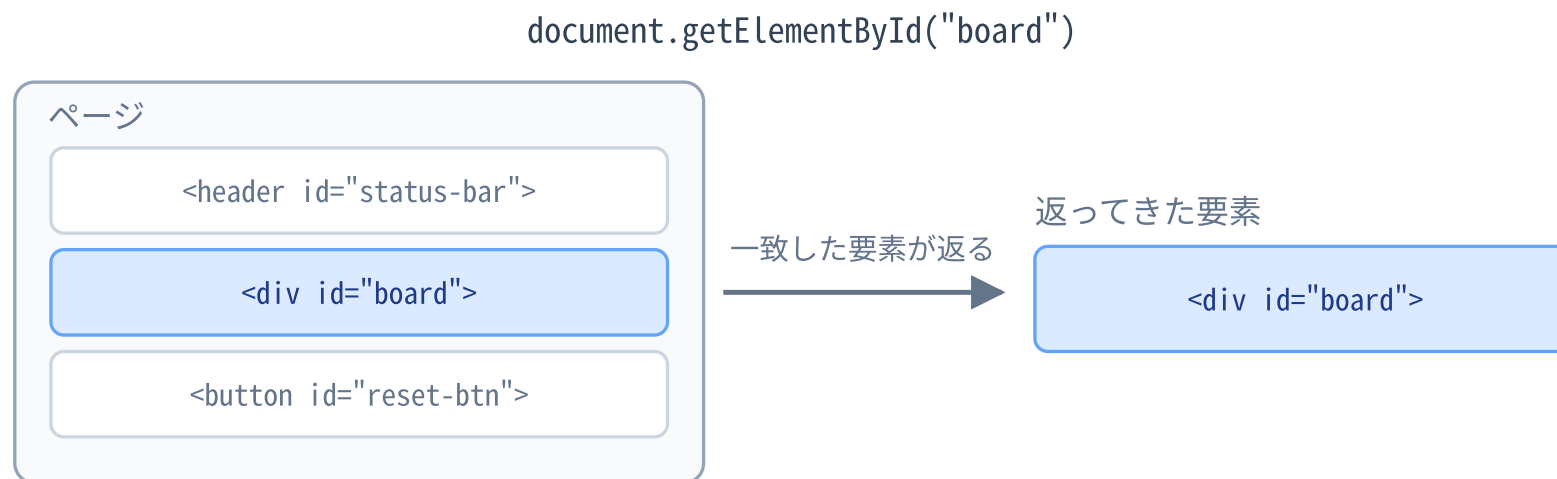
そこで HTML には空の入れ物だけ置いて、中身は JS が作ります。

```
<!-- 配布した index.html。中身は空にしてある -->
<div id="board"></div>
```

1-2. HTML の要素を取得する

入れ物は `index.html` にありますが、中身を作るのは `script.js` です。

JS のコードから `<div id="board">` を触るために、HTML に付けてある `id` を手がかりに要素を取ってきます。



- `document.getElementById(id)` — HTMLの中から `id` 属性がその文字列の要素を探して返す
同じ `id` はページに 1 つだけなので、返るのは 1 要素。見つからなければ `null` が返る

1-2. 盤面を取得する

コピペ `script.js` のいちばん最後に書き足します。

```
// 盤面を取得して boardEl に入れる  
const boardEl = document.getElementById("board");
```

盤面はこのあと何度も触るので、そのたびに

`document.getElementById("board")` と書かずに済むよう、最初
取得して変数にしておきます。

この資料では、HTML 要素を入れる変数の末尾に `El` (Element の略) を付けて統一します。

確認 画面は変わりません。診断パネルの Chapter 1 を開くと
「boardEl が id="board" の要素」に ✓ が付きます。

章ごとのテスト

Chapter 1

4/11

- ✓ deck が配列である
- ✓ deck の長さが 16
- ✓ deck に絵柄が 2 枚ずつある
- ✓ boardEl が id="board" の要素
- × createCard 関数が定義されている
- × createCard('🍎') が Element を返す
- × createCard の返り値に data-symbol が入る
- × createCard の返り値に card クラスが付く
- × createCard の返り値が card > card-inner > front / back の入れ子
- × renderBoard 関数が定義されている
- × 盤面に 16 枚のカードが中身つきで並んでいる

JS で HTML を作る

HTML に書いていない要素も、JS から作って足せます。作る处と、HTML につなぐところの 2 段階です。

1 要素を作る

```
const element = document.createElement(tagName);
```



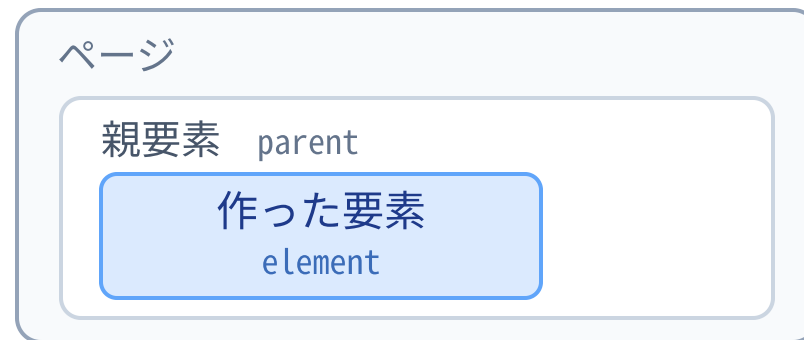
まだ画面に出ない

親要素の中に入る



2 HTML につなぐ

```
parent.appendChild(element);
```



ここではじめて画面に出る

`tagName` は `"div"` や `"p"` のような文字列です。戻り値の `element` は 2 でも使うので変数で受けます。

`parent` は入れ先の要素です。今回の入れ先は盤面なので、さきほど取得した `boardEl` を渡します。

試しに 1 つ作ってみる

コピー `script.js` のいちばん最後に貼ります。

```
const apple = document.createElement("div"); // <div></div> ができる
apple.textContent = "🍏"; // <div>🍏</div> になる
boardEl.appendChild(apple); // 盤面の中に入る
```

貼ると、盤面のところに 🍏 が 1 つ出ます。



盤面の中身が変わった

```
// さっき貼った 3 行
const apple = document.createElement("div");
apple.textContent = "🍏";
boardEl.appendChild(apple);
```

```
<!-- 貼る前 -->
<div id="board"></div>
```

```
<!-- 貼った後 -->
<div id="board">
  <div>🍏</div>
</div>
```

`createElement` で作った要素は、`appendChild` で HTML の中に入れて、はじめて表示されます。

🍏 が 1 つ出るのを確かめたら、この 3 行は消します。

1-3. 作りたいカードの HTML

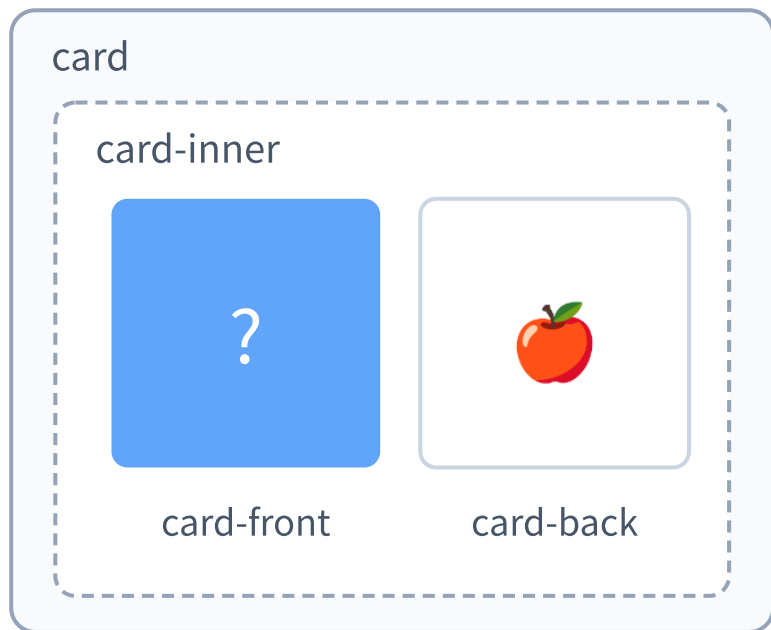
これから `createCard` という関数を書きます。`createCard("🍎")` を呼ぶと、次の HTML が組み立てられて返ってくる形にします。

この HTML はどこにも書きません。出来上がりを確認するためだけのものです。

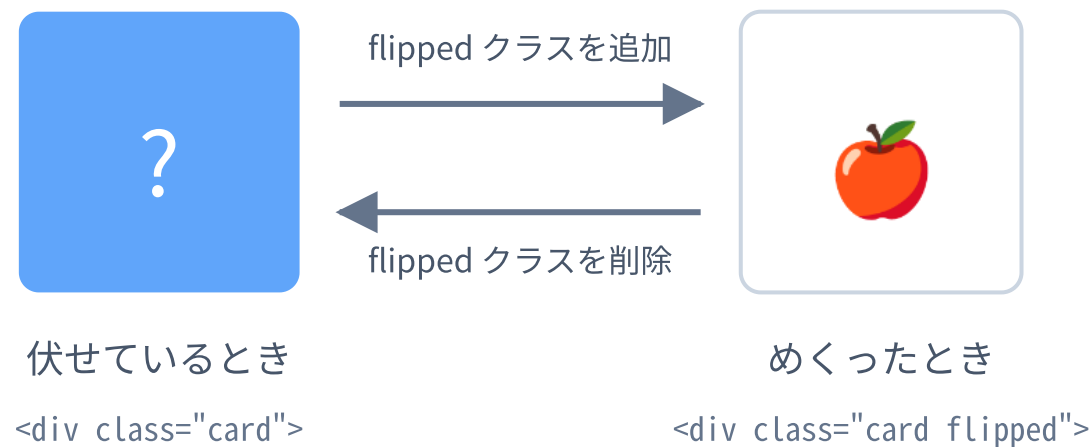
```
<div class="card" data-symbol="🍎">
  <div class="card-inner">
    <div class="card-front">?</div>
    <div class="card-back">🍎</div>
  </div>
</div>
```

1-3. カードの構造

HTML の入れ子



画面での見え方



カードの向きは、JavaScript で `card` に `flipped` クラスを追加・削除して切り替えます。

1-3. JS と HTML の対応

JS で書いた 1 行が、HTML のどこになるかの対応です。

やること	JS	できる HTML
要素を作る	<code>document.createElement("div")</code>	<code><div></div></code>
クラスを追加する	<code>el.className = "card"</code>	<code><div class="card"></code>
データを持たせる	<code>el.dataset.symbol = "🍎"</code>	<code><div data-symbol="🍎"></code>
文字を入れる	<code>el.textContent = "?"</code>	<code><div>?</div></code>
要素の中に入れる	<code>parent.appendChild(child)</code>	<code><parent><child></child></parent></code>

`document.createElement` で作っただけの要素は、まだどこにも属していません。`appendChild` でつなげて初めて入れ子になります。

<button class="timer-btn" data-delta="-60">-</button> — <button class="timer-btn" data-delta="60">+</button>

1-3. カード 1 枚を作る関数

記述 コードブロックをそのまま `script.js` に貼って、【A】～【C】を書き換えましょう。

候補: `card` / `inner` / `front` / `back`

```
// 内側の要素から順に、どこに入れるかを考える
function createCard(symbol) {
  const card = document.createElement("div");
  card.className = "card";
  card.dataset.symbol = symbol;

  const inner = document.createElement("div");
  inner.className = "card-inner";
  const front = document.createElement("div");
  front.className = "card-front";
  front.textContent = "?";
  const back = document.createElement("div");
  back.className = "card-back";
  back.textContent = symbol;

  【A】.appendChild(front);
  【B】.appendChild(back);
  【C】.appendChild(inner);
  return card;
}
```

確認 診断パネルの Chapter 1 を開くと、入れ子を見ているテストに✓が付きます。次のスライドで答え合わせをします。

章ごとのテスト

Chapter 1

9/11

- ✓ deck が配列である
- ✓ deck の長さが 16
- ✓ deck に絵柄が 2 枚ずつある
- ✓ boardEl が id="board" の要素
- ✓ createCard 関数が定義されている
- ✓ createCard('🍎') が Element を返す
- ✓ createCard の返り値に data-symbol が入る
- ✓ createCard の返り値に card クラスが付く
- ✓ createCard の返り値が card > card-inner > front / back の入れ子
- ✗ renderBoard 関数が定義されている
- ✗ 盤面に 16 枚のカードが中身つきで並んでいる

1-3 答え合わせ

```
// ... 要素を作るところは省略 ...  
  
inner.appendChild(front);    // A: card-front が card-inner の内側に入る  
inner.appendChild(back);     // B: card-back も card-inner の内側に入る  
card.appendChild(inner);     // C: card-inner が card の内側に入る  
  
return card;  
}
```

作りたいカードの HTML と見比べると、内側の要素から順に入れていることが分かります。

1-3 の確認: 1 枚だけ出してみる

コピペ 作った関数を 1 回だけ呼びます。

```
boardEl.appendChild(createCard("🍎"));
```

伏せた状態のカード (青い「?」) が 1 枚出れば OK です。確認できたらこの 1 行は消します。

何も出ないときは、【A】～【C】がまだ入れ子になっていません。前のスライドの答えと見比べて直しましょう。



1-3 補足: カードから絵柄を取り出す

Chapter 3 では、めくった 2 枚が同じ絵柄かを判定します。そのためには、カード要素から絵柄だけを取り出せる必要があります。

書かれている文字を取得するには `textContent` を使います。

ただし、カードの中には伏せているときに見える `?` と、めくると見える絵柄が両方入っています。

```
<!-- createCard で作ったカードの再掲。書き足すものではない -->
<div class="card">                                <!-- この textContent を読むと -->
  <div class="card-inner">
    <div class="card-front">?</div>               <!-- この ? と -->
    <div class="card-back">🍎</div>               <!-- この 🍎 がつながって返る -->
  </div>
</div>
```

`textContent` は中にある文字を全部つなげて返すので、伏せているときに見える `?` が混ざります。

```
card.textContent // "?🍎"
```

1-3 補足: dataset で絵柄を持たせる

そこで、絵柄をカードの中の文字としてではなく、カード要素自身に持たせます。ここで `dataset` という仕組みを使います。

```
card.dataset.symbol = "🍎";    // 要素に data-symbol="🍎" が付く  
card.dataset.symbol        // "🍎" - 絵柄だけ取れる
```

`dataset` を使うと、HTML の要素に自分で決めた名前でデータを紐付けられます。今回の場合なら `symbol` の部分がその名前です。

<button class="timer-btn" data-delta="-60">-</button> — <button class="timer-btn" data-delta="60">+</button>

1-4. 盤面全体を描く関数

`deck` の各要素をカードにして盤面に並べる `renderBoard` 関数を書きます。

- `element.replaceChildren()` — その要素の中身を全部削除
例: `boardEl.replaceChildren()` → 盤面に並んだカードが消える

記述 コードブロックをそのまま `script.js` に貼って、【A】～【B】を書き換えましょう。

候補: `createCard` / `renderBoard` / `appendChild` / `replaceChildren`

```
// deck の各要素をカードにして盤面に並べる
function renderBoard() {
  boardEl.replaceChildren(); // 中身を全部削除

  // deck の要素を 1 つずつ取り出して繰り返す
  for (let i = 0; i < deck.length; i++) {
    const card = 【A】(deck[i]);
    boardEl. 【B】 (card);
  }
}

renderBoard();
```

確認 16 枚のカードが 4×4 に並びます。「盤面 (DOM)」のカード枚数も 16 になります。

1-4 答え合わせ

```
function renderBoard() {  
  boardEl.replaceChildren(); // 中身を全部削除  
  
  // deck の要素を 1 つずつ取り出して繰り返す  
  for (let i = 0; i < deck.length; i++) {  
    const card = createCard(deck[i]); // A  
    boardEl.appendChild(card);        // B  
  }  
}  
  
renderBoard();
```

1 枚作って 1 枚並べる、を `deck` の要素数だけ繰り返します。 `createCard` が返してくるのはカード 1 枚の要素なので、そのまま `appendChild` に渡せます。

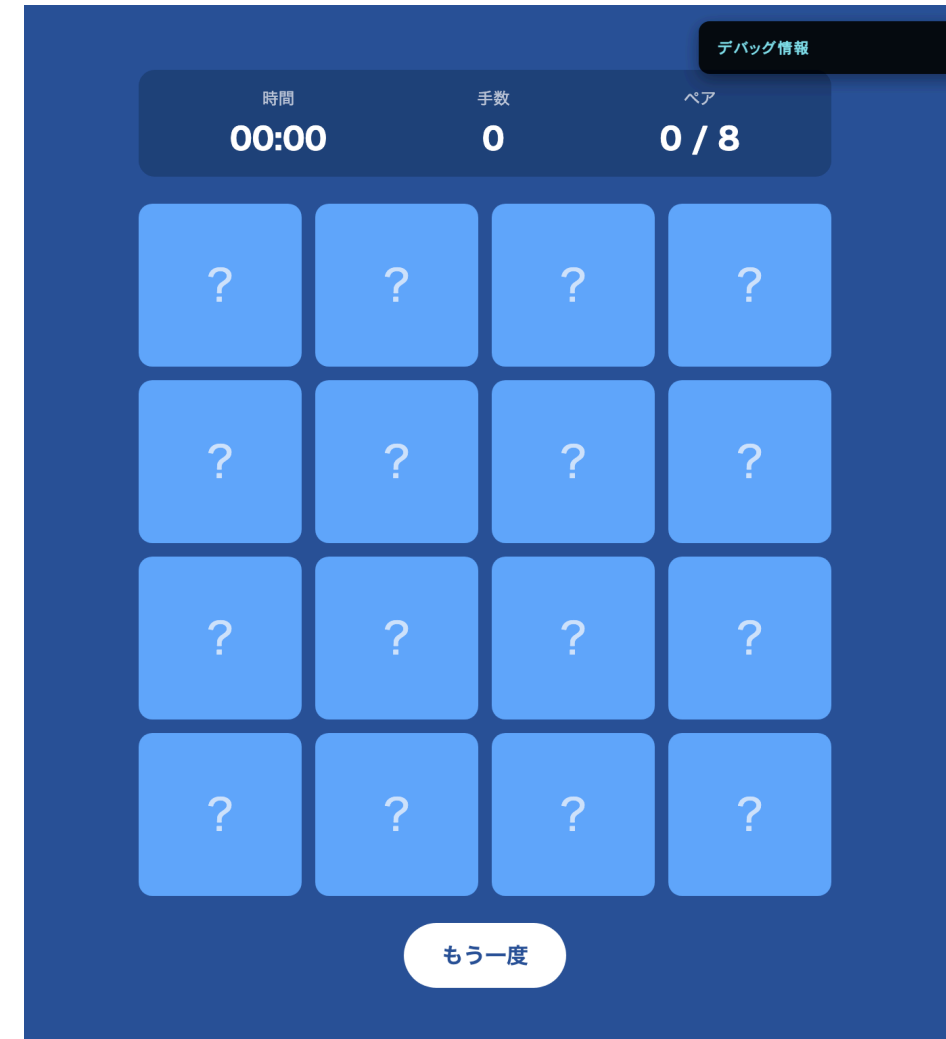
Chapter 1 チェックポイント

- プレビューに 16 枚のカードが 4×4 に並んでいる
- カードは全部伏せた状態 (青い「?」) で並んでいる
- 診断パネルの Chapter 1 が 11/11 ✓ になっている

動かないときは:

- Console にエラーが出ていないか
- `renderBoard()` を呼び忘れていないか
- `symbols.concat(symbols)` のドットを忘れていないか

追いつき用: `ch1.js`



休憩 (5 分)

`<button class="timer-btn" data-delta="-60">-</button>`

`<button class="timer-btn" data-delta="60">+</button>`

まだ 16 枚並んでいない人は、この間に `ch1.js` で追いつけます。詰まったところはチャットで聞いてください。

Chapter 2

クリックでカードをめくる

Chapter 2 のゴール

カードをクリックするとめくれるようにします。ただし次のルールを守ります。

- すでにめくったカードは再クリックしても反応しない
- 2枚めくったあと、絵柄を比べている間はクリックしてもめくれないようにする (あとで比べる処理を入れるときに使う)

クリックしたカード = オレンジ

伏せているカードをクリック



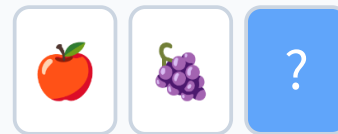
めくれる

めくったカードを再クリック



何も起きない

2枚めくって絵柄を比べる間に
3枚目をクリック



何も起きない

クリック時に必要な 3 つの情報

どちらのルールも、クリックされた時点で次のことが分かっていないと判定できません。

- そのカードはもうめくれているか
- 2 枚めくり終えて、絵柄を比べている最中か (違っていた 2 枚は少しのあいだ見せてから伏せに戻すので、その待ち時間のこと)
- めくったのが 1 枚目か 2 枚目か (2 枚目なら比べる番になるため)

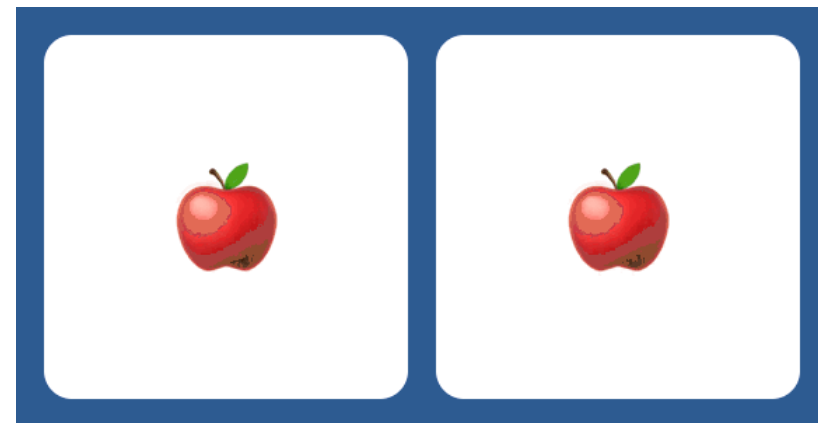
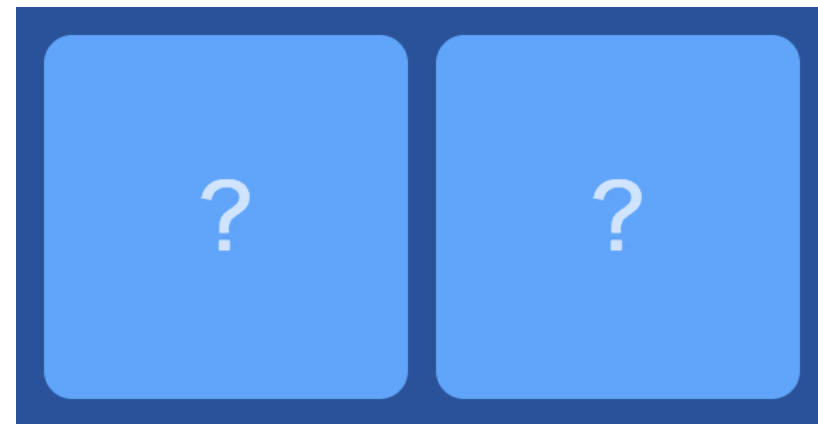
今どうなっているかを保持しておく変数を状態変数と呼びます。この章では状態変数を用意し、それを見て「めくるか、何もしないか」を決める関数を書き、カードに紐づけていきます。

配布 CSS の約束

CSS 側は次のように書かれています。JS 側はクラスを付けるだけで見た目が動きます。

- `flipped` — カードがめくれるアニメーションが再生される。
クリックしてカードをめくるときに付ける
- `matched` — 緑色でハイライトされる。2 枚の絵柄が揃ったときに付ける

`flipped` はこの章で、`matched` は Chapter 3 で付けます。



クラスの付け外し

クラスの足し引きに使うのが `classList` です。 `className` のほうは `class` 属性を丸ごと置き換える書き方なので、めくるときに使うと `card` が消えてしまいます。

```
card.className = "flipped";    // class="flipped" になり、card が消える
card.classList.add("flipped"); // class="card flipped" になる
```

要素を 0 から組み立てるときは `className`、すでにあるクラスに足し引きするときには `classList` を使います。

`classList` でできるのは次の 3 つです。

```
card.classList.add("flipped");    // 付ける
card.classList.remove("flipped"); // 外す
card.classList.contains("flipped"); // 付いていれば true、なければ false
```

2-1. 状態変数を用意する

「1枚目にめくったカード」「2枚目にめくったカード」「ロック中か」の3つを変数で持ちます。

コピペ いちばん最後ではなく、`const deck` の行のすぐ下に追加します。

```
// めくりの状態を持つ変数を用意する
let firstCard = null;    // 1枚目にめくったカード
let secondCard = null;  // 2枚目にめくったカード
let lockBoard = false;  // 2枚めくったあとに他のカードを押させないためのロック
```

- `let` — あとで別の値を入れ直すのでこちら。 `const` は入れ直すとエラーになる (詳しくは末尾の付録)

確認 診断パネルの「変数など」で、`firstCard` と `secondCard` が `null`、`lockBoard` が `false` になります。3つとも「(未定義)」から変われば貼れています。

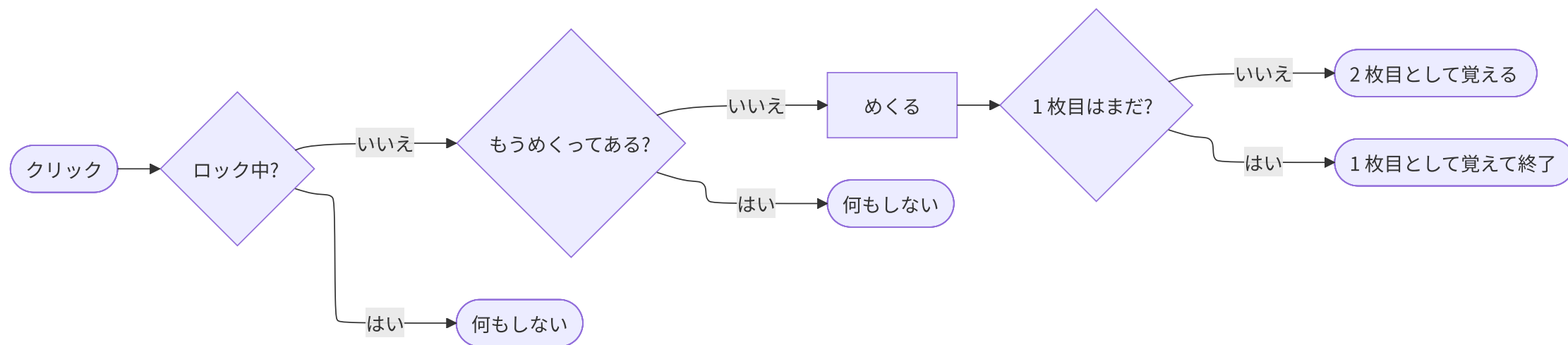
デバッグ情報

コピー ▼

変数など

```
deck          = [Array length=16]
firstCard     = null
secondCard    = null
lockBoard     = false
moves         = (未定義)
matchedPairs  = (未定義)
timerId       = (未定義)
```

2-2. クリック処理を関数にする (仕組み)



図中の日本語ラベルを、2-1 で用意した状態変数や「配布 CSS の約束」のクラス名に置き換えるとコードになります。先にカードと繋いでから、この図を見ながら書きます。

最初の「ロック中?」が実際に効くのは Chapter 3 です。この章のあいだは必ず「いいえ」に進みます。

2-2. 先にカードと繋いでおく

`element.addEventListener("click", 関数)` で、その要素がクリックされたときに実行する関数を紐づけられます。呼ばれる側の `handleCardClick` はまだ作らず、この 1 行だけ先に置きます。こうしておくで、次のスライドで穴を 1 つ埋めるたびに画面で確かめられます。

コピペ `createCard` 関数の中の `return card;` の直前に置きます。

```
function createCard(symbol) {  
  // ... (ここまでに書いた中身は省略) ...  
  card.appendChild(inner);  
  
  card.addEventListener("click", () => handleCardClick(card));  
  
  return card;  
}
```

確認 カードをクリックすると、診断パネルの「診断」に  `Uncaught ReferenceError: handleCardClick is not defined` が出ます。呼ぶ先をまだ作っていないので、これが出れば繋がっています。押すたびに (×2) (×3) と数が増えます。診断パネルの Chapter 2 は 3/8 です。

<button class="timer-btn" data-delta="-60">-</button> — <button class="timer-btn" data-delta="60">+</button>

2-2. クリック処理を関数にする (コード)

記述 ファイルの末尾にこの形で書き、さきほどのフロー図を見ながら【A】～【D】を埋めます。

候補: lockBoard / firstCard / secondCard / "flipped" / "matched" / add / remove

```
// フロー図に沿って書く
function handleCardClick(card) {
  if (【A】) return;
  if (card.classList.contains(【B】)) return;

  card.classList. 【C】 (【B】);

  if (【D】 === null) {
    firstCard = card;
    return;
  }

  secondCard = card;
}
```

▶ ヒント

確認 カードがめくれるようになり、もう一度押しても反応しません。診断パネルの「変数など」で firstCard が card(🍎 @3) に変わります。【A】はクリックでは確かめられないので、同じパネルの Chapter 2 が 8/8 ✓ になるかで見ます。

2-2 答え合わせ

```
function handleClick(card) {  
  if (lockBoard) return; // A  
  if (card.classList.contains("flipped")) return; // B  
  
  card.classList.add("flipped"); // C  
  
  if (firstCard === null) { // D  
    firstCard = card;  
    return;  
  }  
  
  secondCard = card;  
}
```

`classList` は `contains` で調べて `add` で付けます。付いた `"flipped"` に配布 CSS が反応して、めくれるアニメーションが流れます。

`lockBoard` が `true` の間は先頭で打ち切るので、絵柄を比べている最中のクリックはめくる処理まで進みません。`lockBoard` は真偽値なのでそのまま `if` に置けます。`firstCard` に入るのはカードか `null` なので、`null` と比べて「1枚目がまだ無い」を真偽値にしてから置きます。

2-2 補足: カッコがいつ効くか

`addEventListener` に預けるのは、クリックされたときに実行してほしい処理です。カッコを付けて `handleCardClick(card)` と書くと、その場での呼び出しになります。



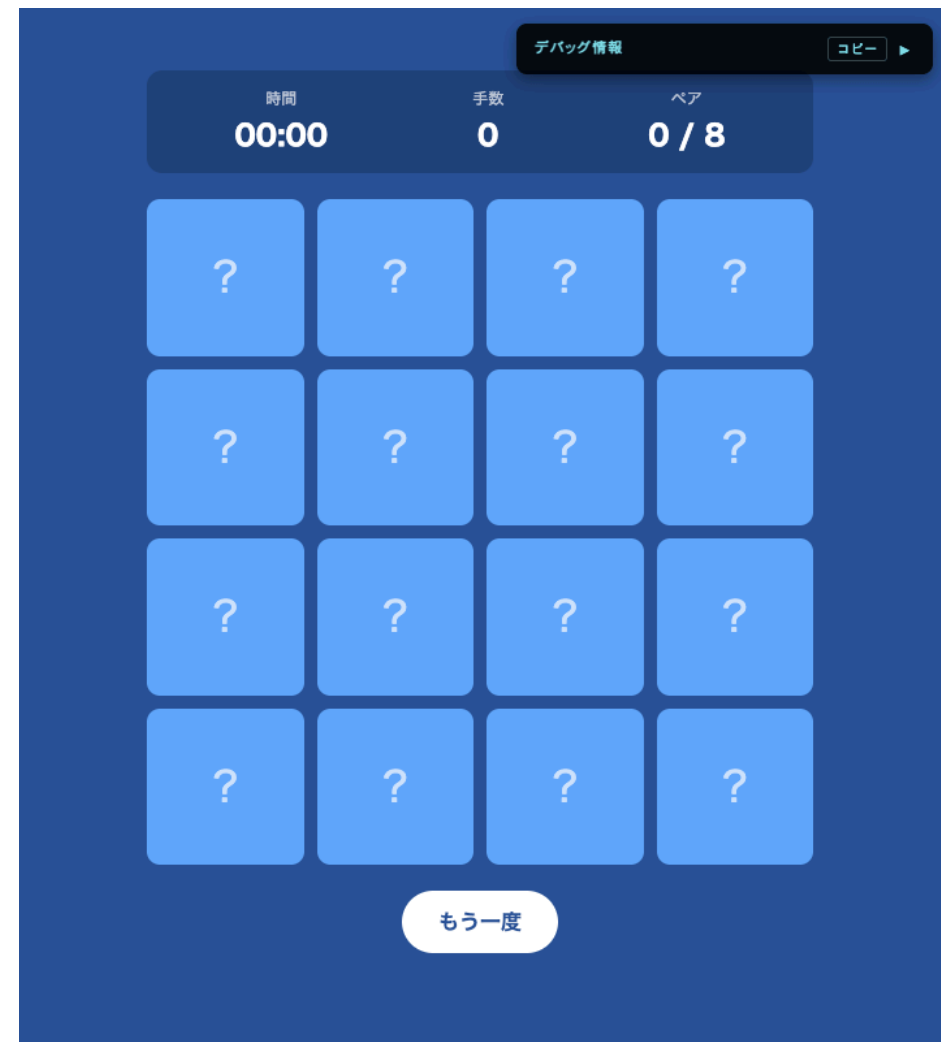
×を試すと16枚すべてが最初からめくれ、あとは何も起きません。✓の `() => handleCardClick(card)` は「`card` を渡して `handleCardClick` を呼ぶ」という処理で、内側のカッコが効くのは実行の瞬間です。

預ける相手には呼ばずに渡し、渡したいものがあるときだけ `() =>` (アロー関数) で包む。

Chapter 2 チェックポイント

- カードをクリックするとめくれて絵柄が見える
- 一度めくったカードは、二度目のクリックでは反応しない
- 3枚目以降もめくれてしまう (これは一致判定を入れるときに止めます)
- 診断パネルの Chapter 2 が 8/8 ✓ になっている

追いつき用: `ch2.js`



Chapter 3

2 枚めくって一致判定

Chapter 3 のゴール

ペア判定を作ります。

- 2枚めくったら、絵柄が同じか比べる
- 同じなら `matched` にして残す
- 違うなら少し待って伏せに戻す
- 判定待ちの間は他のカードを反応させない

3-1. handleClick に判定分岐を追加

コピー handleClick の中、secondCard = card; の下に貼ります。

```
secondCard = card;

// 2 枚目がめくれたら判定する
const isMatch = firstCard.dataset.symbol === secondCard.dataset.symbol;

if (isMatch) {
  handleMatch();
} else {
  handleMismatch();
}
```

比べているのは、1-3 で `card.dataset.symbol = symbol` と入れておいた絵柄です。2 枚のカードから同じ名前を取り出し、`===` で突き合わせます。`===` と `==` の違いは末尾の付録に回します。

確認 2 枚めくると、絵柄が違えば `handleMismatch is not defined`、同じなら `handleMatch is not defined` が出ます。呼び出しがここまで届いた印なので、この時点ではこれで正解です。呼ばれる側は 3-3 と 3-4 で作ります。

<button class="timer-btn" data-delta="-60">-</button> — <button class="timer-btn" data-delta="60">+</button>

3-2. ターンの片付け

一致でも不一致でも、2枚判定した後は「次のターンを迎える」ための後片付けが必要です。3-1 で呼び分けた `handleMatch` と `handleMismatch` は、どちらも最後にこれを通ります。共通で使うものなので先に作ります。

やることは、めくりの状態を持つ3つの変数を、それぞれ宣言したときの値に戻すことです。

記述 コードブロックをそのまま `script.js` に貼って、【A】～【B】を書き換えましょう。

候補: `null` / `false` / `true` / `0`

```
// 一致でも不一致でも、次のターンに向けて状態を戻す
function resetTurn() {
  firstCard = 【A】;
  secondCard = 【A】;
  lockBoard = 【B】;
}
```

確認 まだどこからも呼んでいないので画面は変わりません。診断パネルの Chapter 3 で「resetTurn 関数が定義されている」に✓が付きます。

3-2 答え合わせ

```
function resetTurn() {  
  firstCard = null;      // A  
  secondCard = null;  
  lockBoard = false;     // B  
}
```

一致でも不一致でも同じ後片付けを行うので関数にまとめておくと、変更が必要になっても 1 箇所ですみます。この関数はリセット機能を作るときにも再利用します。

書けた人へ: `lockBoard = false;` の行を消すとどうなるか予想してから、3-3 まで進めた状態で試してみてください。「判定待ちのフラグ」が戻らないと、以降のクリックがすべて弾かれます。

3-3. 一致しなかったときの処理

コピペ

```
// 一致しなかったときは 800ms 待って伏せに戻る
function handleMismatch() {
  lockBoard = true;
  setTimeout(unflipCards, 800);
}

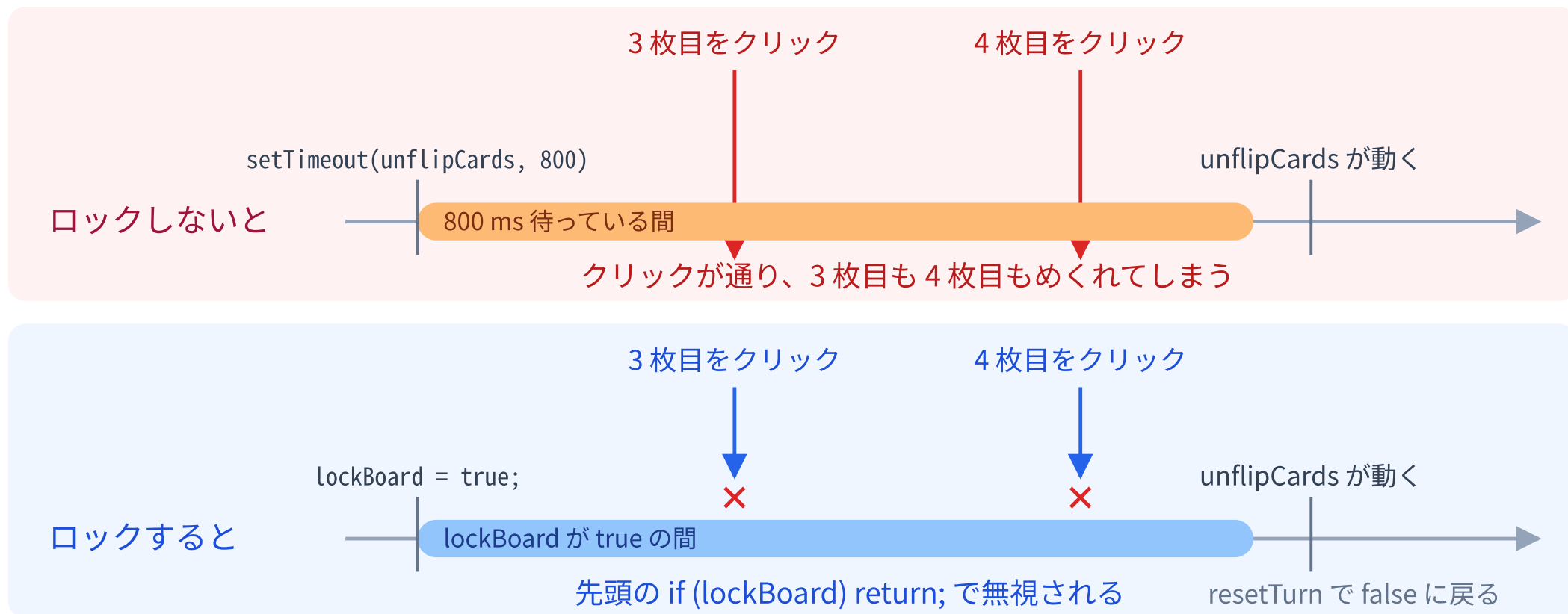
function unflipCards() {
  firstCard.classList.remove("flipped");
  secondCard.classList.remove("flipped");
  resetTurn();
}
```

- `remove` — めくるときに付けた `flipped` を外すメソッド。外すと CSS がカードを伏せた状態に戻る
- `setTimeout(関数, ミリ秒)` — 指定時間後にその関数を 1 回だけ実行する

確認 違う絵柄を 2 枚めくると 800 ms 後に伏せに戻り、そのまま次の 2 枚をめくれます。「変数など」の `lockBoard` が `true` になり、伏せに戻ると `false` に戻ります。同じ絵柄を引いたときは `handleMatch is not defined` が出ます。3-4 で作れば消えます。

なぜ `lockBoard = true` するのか

`setTimeout(関数, 800)` は、ブラウザに「800 ms 後にこれを読んで」と関数を預けて、すぐ次の行に進みます。2-2 の `addEventListener` で関数を預けたのと同じ形で、呼ぶきっかけがクリックから時間に変わっただけです。預けた関数が後から呼ばれるこの動きを非同期と呼びます。



<button class="timer-btn" data-delta="-60">-</button> — <button class="timer-btn" data-delta="60">+</button>

3-4. 一致したときの処理

3-1 で `handleMatch()` を呼ぶところまでは書けています。呼ばれる側を作ります。

記述 コードブロックをそのまま `script.js` に貼って、【A】 ～ 【B】 を書き換えましょう。

候補: `add` / `remove` / `flipped` / `matched`

```
// 一致した 2 枚を光らせて、次のターンに向けて片付ける
function handleMatch() {
  firstCard.classList.【A】 ("【B】 ");
  secondCard.classList.【A】 ("【B】 ");
  resetTurn();
}
```

`resetTurn()` は 3-2 で書いた後片付けです。ここでは呼ぶだけです。

確認 同じ絵柄を 2 枚めくると両方が緑に光ったまま残り、続けて次のペアを探せます。「盤面 (DOM)」の `matched` が 2 になり、「変数など」の `firstCard` と `secondCard` は `null` に戻ります。まだシャッフルしていないので、いちばん左上のカードと、その 2 行下のカードが同じ絵柄です。

3-4 答え合わせ

```
function handleMatch() {  
  firstCard.classList.add("matched");    // A は add、B は matched  
  secondCard.classList.add("matched");  
  resetTurn();  
}
```

`matched` クラスが付くと CSS 側が緑に光らせます。

`flipped` を外す処理はここには要りません。めくったまま緑で残すのが、取れたペアの見せ方です。

片付けは 3-2 の `resetTurn` にまとめてあるので、ここでは呼ぶだけで済みます。

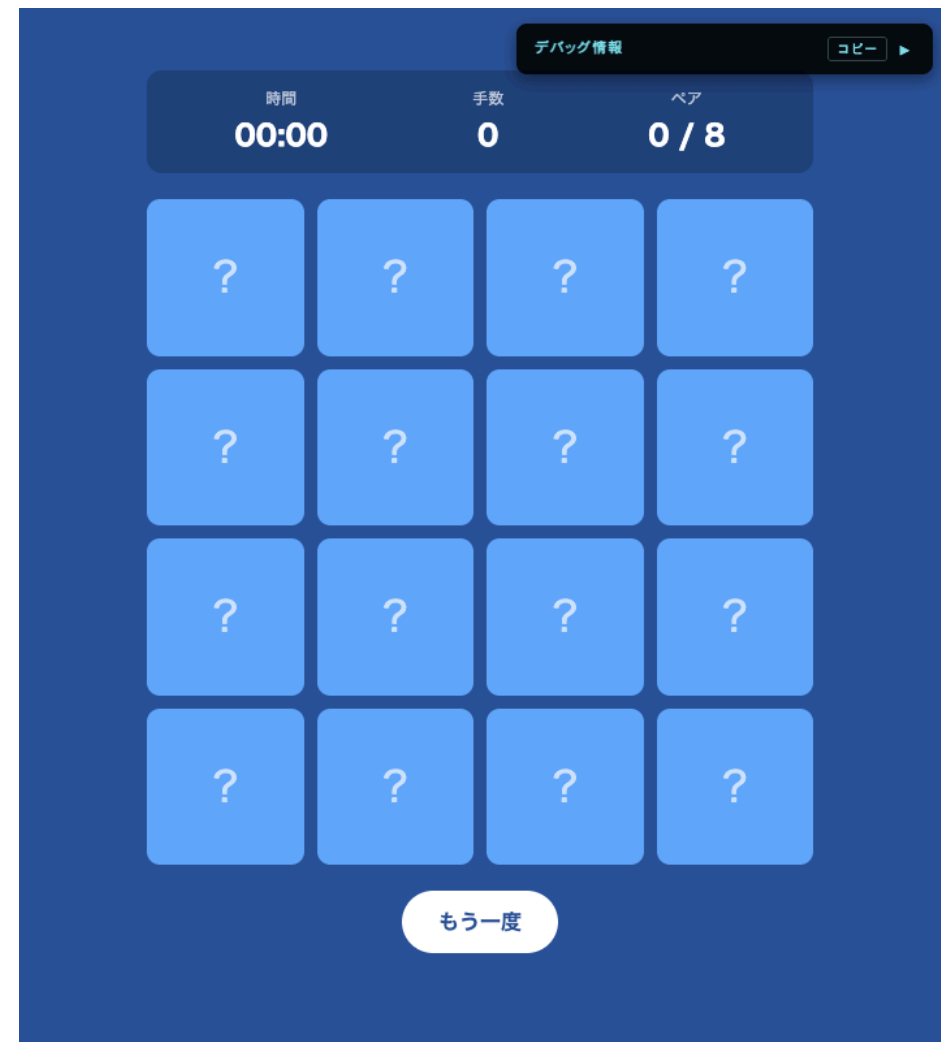
この `handleMatch` には、あとでペア数の更新とクリア判定を足して書き換えます。

Chapter 3 チェックポイント

- 同じ絵柄を 2 枚選ぶと、両方とも緑色に光ったまま残る
- 違う絵柄を選ぶと、少し待って両方伏せに戻る
- 判定待ちの間は他のカードが反応しない
- 診断パネルの Chapter 3 が 4/4 ✓ になっている

動かないときは、Console のエラー行番号と関数名スペルを見てください。 `dataset.symbol` の綴り、 `handleMatch` と `handleMismatch` の呼び分けが typo で入れ替わっていないかチェック。

追いつき用: `ch3.js`



休憩 (10 分)

`<button class="timer-btn" data-delta="-60">-</button>`

`<button class="timer-btn" data-delta="60">+</button>`

後半はシャッフル・タイマー・手数・リセットを足して、遊べる状態に仕上げます。

Chapter 4

カードをシャッフル

Chapter 4 のゴール

今のままだとカードの並びが毎回同じなので、配置をランダムにします。

シャッフルには Fisher-Yates という定番アルゴリズムを使います。よく見かける `array.sort(() => Math.random() - 0.5)` は、実は統計的にきれいに混ざりません。詳しくは末尾の付録に回します。

4-1. Fisher-Yates シャッフル

後ろから前へ走査し、`i` と `0 ~ i` の中のランダムな位置を入れ替える定番アルゴリズムです。渡された配列はそのままに、コピーを混ぜて返します。

コピペ

```
function shuffle(array) {  
  const result = array.slice();  
  for (let i = result.length - 1; i > 0; i--) {  
    const j = Math.floor(Math.random() * (i + 1));  
    [result[i], result[j]] = [result[j], result[i]];  
  }  
  return result;  
}
```

確認 まだ呼んでいないので画面は変わりません。診断パネルが `shuffle` を実際に呼んで調べるので、Chapter 4 が 4/4 ✓ になれば合っています。

4-2. deck をシャッフルする

`deck` の宣言を書き換えます。 `const` から `let` に変わっている点に注意してください。

コピペ

```
// 変更前
const deck = symbols.concat(symbols);

// 変更後
let deck = shuffle(symbols.concat(symbols));
```

あとで作るリセット機能でも `deck` に新しい配列を入れ直します。 `const` のままだと再代入で `TypeError` になるので `let` に変えます。4-1 の `result` のように中身を書き換えるだけなら `const` のままで通ります。違いは末尾の付録に回します。

確認 リロードするたびにカードの並びが変わります。

Chapter 4 チェックポイント

- ブラウザをリロードするたびにカードの並びが変わる
- ペア (同じ絵柄) はちゃんと 2 枚ずつ含まれている
- 診断パネルの Chapter 4 が 4/4 ✓ になっている

追いつき用: `ch4.js`

Chapter 5

タイマー・手数・クリア判定

Chapter 5 のゴール

画面上部の 3 つの表示 (時間・手数・ペア) を動かし、クリア判定を入れます。

- 時間: 1 枚目をめくった瞬間から計測開始
- 手数: 2 枚目をめくるたびに +1
- ペア: ペアが揃うたびに更新
- 全ペア揃ったらクリアメッセージ表示 + タイマー停止

5-1. 状態変数と要素の参照を追加

コピペ いちばん最後ではなく、`let lockBoard = false;` の下にまとめて追加します。

```
let lockBoard = false;

// 手数、ペア数、タイマー用の状態を用意
let moves = 0;           // 2 枚めくるごとに 1 増える手数
let matchedPairs = 0;    // 揃ったペアの数
let timerId = null;      // 動いているタイマーの番号。止めるときに使う
let startTime = 0;       // 計測を始めた時刻

const timerEl = document.getElementById("timer");
const movesEl = document.getElementById("moves");
const pairsEl = document.getElementById("pairs");
const clearMessageEl = document.getElementById("clear-message");
```

状態変数は 1 箇所に集めておくと、リセット機能を作るときに「何を戻せばよいか」が一目で分かります。

確認 「変数など」に `moves = 0`、`matchedPairs = 0`、`timerId = null` が出ます。診断パネルの Chapter 5 は 7/10 まで進みます。

<button class="timer-btn" data-delta="-60">-</button> — <button class="timer-btn" data-delta="60">+</button>

5-2. 手数を更新

書く場所は `handleCardClick` の中、`secondCard = card;` と `const isMatch = ...` の間です。この 2 行が「状態を変えたら描画を更新する」という型になります。

記述 前後の既存行を目印に、【A】～【B】を埋めた 2 行を書き足しましょう。

候補: `moves` / `movesEl` / `matchedPairs` / `pairsEl` / `timerEl`

```
secondCard = card;
```

```
【A】 ++;
```

```
【B】 .textContent = 【A】 ;
```

```
const isMatch = firstCard.dataset.symbol === secondCard.dataset.symbol;
```

- `x++` — `x = x + 1` の短縮形
- `element.textContent = 値` — その要素の表示テキストを書き換える

確認 2 枚めくるたびに画面上部の「手数」が 1 ずつ増えます。「変数など」の `moves` も同じ数になります。

5-2 答え合わせ

`handleCardClick` の中、`secondCard = card;` の直後、判定の前です。

```
secondCard = card;

moves++;           // A は状態変数の moves
movesEl.textContent = moves; // B は表示先の要素 movesEl

const isMatch = firstCard.dataset.symbol === secondCard.dataset.symbol; // 判定はすでにある行
```

`moves++` は `moves = moves + 1` の短縮形。状態を +1 したら、その直後に描画を更新する。この 2 行 1 セットが、今日の講座で一番繰り返される型です。

5-3. タイマーの開始と停止

コピペ そのまま貼って OK。

```
function startTimer() {
  startTime = Date.now();
  // 1000 ms 間隔だと秒表示のズレが目立つので少し細かめに回す
  timerId = setInterval(renderTimer, 250);
}

function renderTimer() {
  const elapsed = Math.floor((Date.now() - startTime) / 1000);
  const mm = String(Math.floor(elapsed / 60)).padStart(2, "0");
  const ss = String(elapsed % 60).padStart(2, "0");
  timerEl.textContent = `${mm}:${ss}`;
}

function stopTimer() {
  clearInterval(timerId);
  timerId = null;
}
```

- `setInterval(関数, ミリ秒)` — 一定間隔でその関数を繰り返し実行する
- `timerId` — `setInterval` が返す識別子。動いているものを直接は掴めないで、止めるにはこの番号が要る
- `startTime` — `Date.now()` で取った開始時刻 (ミリ秒)。経過秒は差を 1000 で割って出す
- `renderTimer` — 250 ms ごとに呼ばれ、経過秒を計算し直して表示を書き換える関数
- ``${mm}:${ss}`` — テンプレートリテラル。変数を埋め込める

`setInterval` は `clearInterval` を呼ぶまで止まりません。不要になったら必ず止めます (`stopTimer` の役割)。

確認 呼び出しをまだ書いていないので、動かして確かめる方法はありません。5-4 を入れてから確認します。

5-4. タイマー開始を組み込む

コピペ `handleCardClick` の `if (firstCard === null)` の分岐に 1 行追加します。

```
// 1 枚目をめくった瞬間にタイマー開始（まだ動いていなければ）
if (firstCard === null) {
  firstCard = card;
  if (timerId === null) startTimer();
  return;
}
```

`timerId === null` は「まだタイマーが動いていない」ということです。2 枚目、3 枚目のクリックでは既に ID が入っているので、`startTimer` は呼ばれず、最初の 1 回だけ動きます。

確認 1 枚目をめくった瞬間から時間が動き出します。「変数など」の `timerId` が `null` から数値に変わり、2 枚目以降でその数値が変わらなければ二重に動いていません。

<button class="timer-btn" data-delta="-60">-</button> — <button class="timer-btn" data-delta="60">+</button>

5-5. handleMatch にペア数とクリア判定を追加

記述 handleMatch を書き換えます。【A】 ～ 【C】 を埋めましょう。

【B】 【C】 の候補: symbols.length / deck.length / 8 / stopTimer() / startTimer()

```
function handleMatch() {  
  firstCard.classList.add("matched");  
  secondCard.classList.add("matched");  
  // ペア数を 1 増やして、その場で表示も更新する 2 行。表示は「3 / 8」の形  
  【A】  
  resetTurn();  
  if (matchedPairs === 【B】) {  
    【C】 ;  
    clearMessageEl.textContent = `クリア! ${moves}手 / ${timerEl.textContent}`;  
  }  
}
```

確認 ペアを取るたびに「ペア」の表示が 1 / 8 のように増えます。

▶ ヒント

5-5 答え合わせ

```
// A (2 行)
matchedPairs++;
pairsEl.textContent = `${matchedPairs} / ${symbols.length}`;
// ...
if (matchedPairs === symbols.length) { // B
    stopTimer();                       // C
}
```

揃えるべきペア数は `symbols.length` から取ります。8 と直接書いても動きますが、絵柄の種類を変えると判定が追従しません。マジックナンバーを避けて由来のある値を使うのは、可読性を上げる基本的な習慣です。

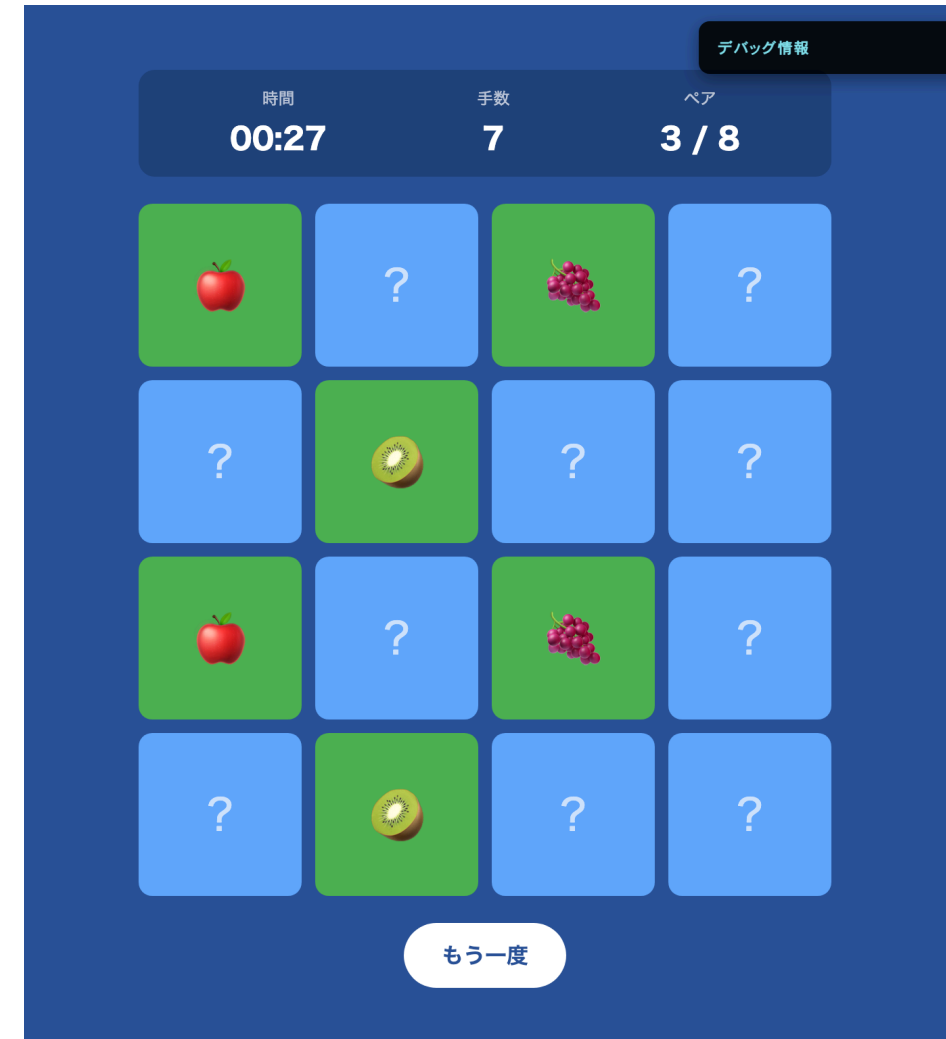
`stopTimer()` を呼ばないと、クリアしたあともタイマーだけ動き続けます。`setInterval` は `clearInterval` を呼ぶまで止まりません。

状態を変えたら、その場で描画も更新する。この 2 行 1 セットは今日 2 回目で、リセット機能でも同じ形が出てきます。

Chapter 5 チェックポイント

- 1枚目をめくった瞬間からタイマーが動き出す
- 2枚目をめくるたびに手数が+1される
- ペアを取るたびにペア数が更新される
- 全ペア取ったらクリアメッセージが出て、タイマーが止まる
- 診断パネルの Chapter 5 が 10/10 ✓ になっている

追いつき用: `ch5.js`



Chapter 6

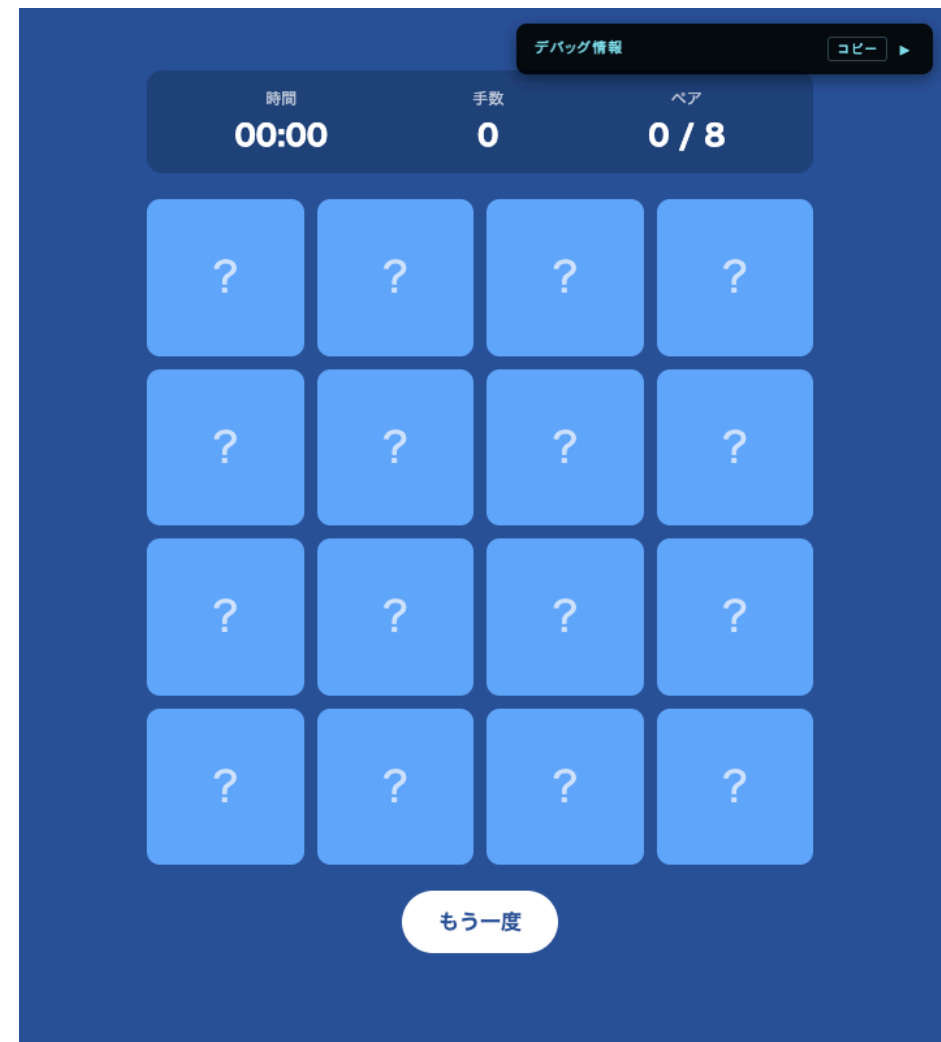
リセット機能

Chapter 6 のゴール

「もう一度」ボタンを機能させて、最初から遊べるようにします。
戻すものは 2 種類です。

- 内部の状態: 盤面の中身、めくりかけのカード、手数とペア数
- 画面の表示: 時間、手数、ペア、クリアメッセージ

あわせて「初回起動」と「リセット」を同じ処理で扱うよう整理します。



<button class="timer-btn" data-delta="-60">-</button> — <button class="timer-btn" data-delta="60">+</button>

6-1. resetGame 関数

記述 状態変数を初期値に戻し、盤面を作り直します。【A】 ～ 【C】 を埋めて、ファイルのいちばん最後に書きます。

候補: `shuffle(symbols.concat(symbols))` / `resetTurn()` / `renderBoard()` / `resetGame()` / `createCard()`

```
// ゲームを初期状態に戻す
function resetGame() {
  stopTimer();
  deck = 【A】 ;
  【B】 ; // めくりの状態に戻す
  moves = 0;
  matchedPairs = 0;
  timerEl.textContent = "00:00";
  movesEl.textContent = "0";
  pairsEl.textContent = `0 / ${symbols.length}`;
  clearMessageEl.textContent = "";
  【C】 ;
}

// 初回の描画も resetGame に任せる
// (renderBoard 関数の下の renderBoard(); は消す)
resetGame();
```

▶ ヒント

確認 リロードするたびに並びが変わります。ここまで動けば【A】と【C】は合っています。

6-1 答え合わせ

```
function resetGame() {  
  stopTimer();  
  deck = shuffle(symbols.concat(symbols)); // A  
  resetTurn(); // B  
  moves = 0;  
  matchedPairs = 0;  
  timerEl.textContent = "00:00";  
  movesEl.textContent = "0";  
  pairsEl.textContent = `0 / ${symbols.length}`;  
  clearMessageEl.textContent = "";  
  renderBoard(); // C  
}
```

めくりの状態を戻す 3 行は `resetTurn` にまとまっているので、ここでは呼ぶだけで済みます。初回起動もリセットも同じ処理で扱えるようになり、状態変数の初期化が一箇所に集約されます。

並び順が効くのは `deck` への代入と `renderBoard()` の関係だけです。`renderBoard()` は呼ばれた時点の `deck` を読むので、代入より前に置くと古い並びで盤面を作ってしまう、「もう一度」を押しても配置が変わりません。`resetTurn()` と表示を戻す 4 行は、この関数の中ならどこに置いても結果は同じです。

<button class="timer-btn" data-delta="-60">-</button> — <button class="timer-btn" data-delta="60">+</button>

6-2. リセットボタンにイベントを付ける

記述 さっき置いた `resetGame();` の前に書き足します。【A】 と 【B】 を埋めましょう。

候補: `getElementById` / `addEventListener`

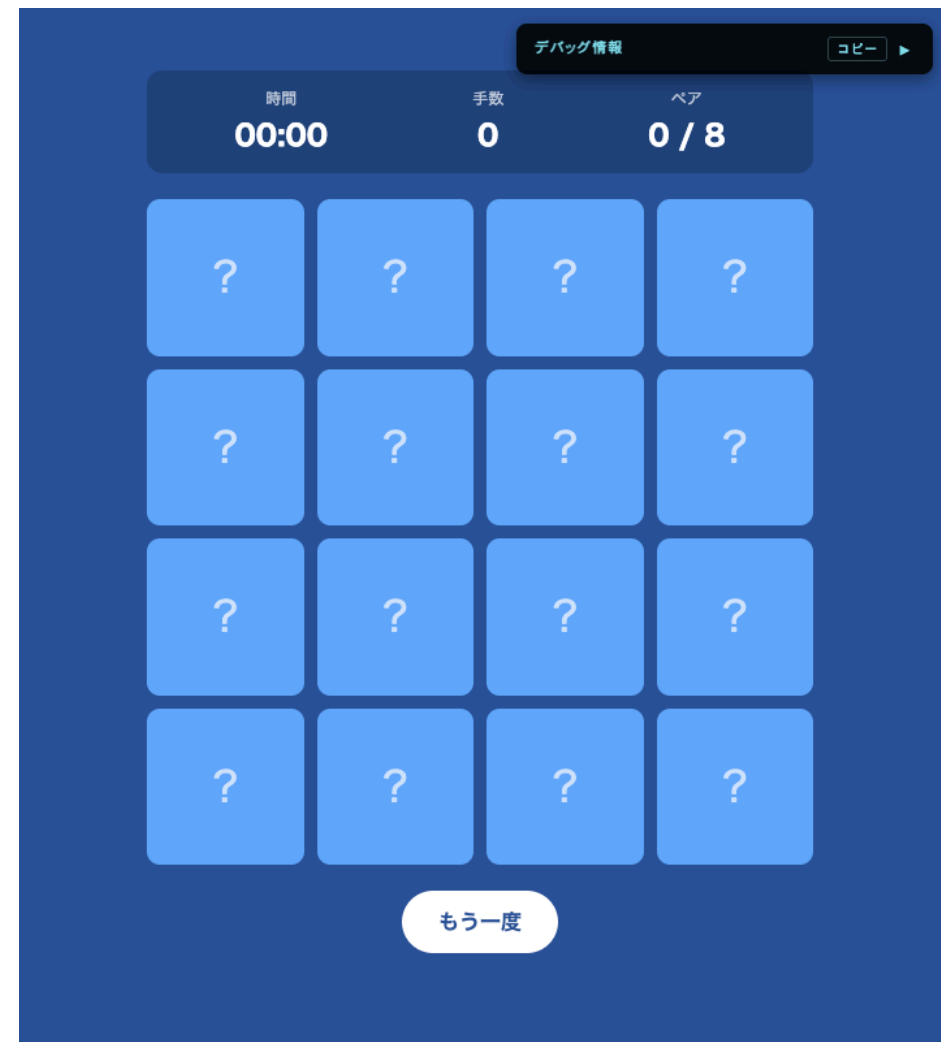
```
// もう一度ボタンで resetGame を呼ぶ
const resetBtn = document. 【A】 ("reset-btn");
resetBtn. 【B】 ("click", resetGame);
```

渡すのは `resetGame` です。カッコを付けて `resetGame()` と書くと、クリック時ではなく `addEventListener` を呼んだ瞬間に実行されてしまいます。2-2 と同じで、預ける相手には呼ばずに渡します。ここは渡したいものがないので、`() =>` で包まずに名前だけを書きます。

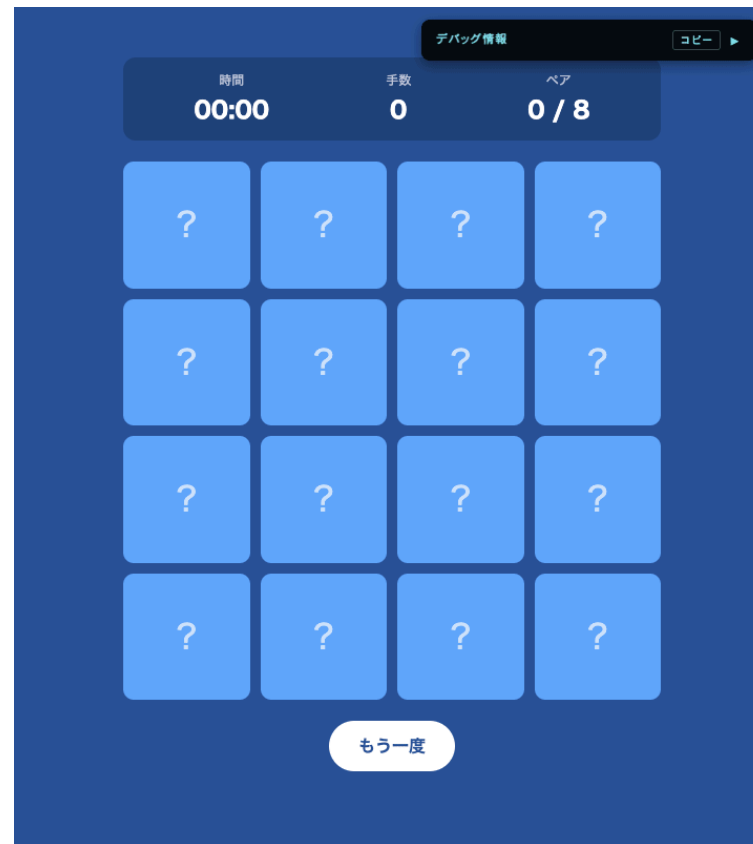
確認 「もう一度」ボタンを押すと並びが変わって最初から始まります。1枚めくった状態で押しても、めくりかけが残らずに遊び直せれば、6-1 の【B】も合っています。【A】と【B】を取り違えると Console に赤いエラーが出ます。

Chapter 6 チェックポイント

- 「もう一度」ボタンを押すと盤面がシャッフルし直されて再スタートする
- ゲーム途中で押してもリセットされる
- タイマー、手数、ペア数、クリアメッセージがすべて初期化される
- 診断パネルの Chapter 6 が 2/2 ✓ になっている



完成



今日やったことの名前

今日書いたものを、名前で並べ直すようになります。

章	書いたもの	名前
1	<code>createCard(symbol)</code> — 引数から画面の部品を 1 つ返す	コンポーネント
2	<code>addEventListener</code> — 呼ぶ相手をブラウザに任せる	制御の反転
2	<code>firstCard</code> , <code>lockBoard</code> — 画面の今を JS 側に持つ	状態変数
6	<code>resetGame()</code> — 何回呼んでも同じ初期状態になる	冪等性

どれも JavaScript 固有の話ではないので、他の言語でも同じ名前で出てきます。覚えて帰らなくても、いつか名前を見たときに「これか」と思い出せば十分です。

おつかれさまでした

今日書いたコードは StackBlitz に残るので、続きはいつでも触れます。

この資料と、応用課題の実装例 (`examples/challenges/`) はこちらにあります。

https://github.com/jigintern/study_session_materials/tree/main/2026/shinkei-suijaku

応用課題の目次

もっとやりたい人向けの応用課題です。着手しやすい順に 10 個、1 個につき 1 スライドで説明します。1 から 3 は数行の書き換え、4 から 6 は少し書き足し、7 から 10 は作りそのものが変わります。

#	応用課題	答えの置き場所
1	伏せるまでの長さを変える	課題文のなか
2	絵柄を変える	課題文のなか
3	ペア数を変える	課題文のなか
4	記憶タイム	課題文のなか
5	ベストスコアを保存する	付録
6	リセットに残るバグを直す	付録
7	ペアの条件を変える	<code>examples/challenges/pair-rule.js</code>
8	難易度切り替え	<code>examples/challenges/difficulty.js</code>
9	カウントダウンモード	<code>examples/challenges/countdown.js</code>
10	状態を state にまとめる	<code>examples/challenges/state.js</code>

応用課題 1: 伏せるまでの長さを変える

3-3 で書いた `handleMismatch` の `800` が、不一致の 2 枚が見えている時間です。単位はミリ秒なので、`800` は 0.8 秒です。

```
function handleMismatch() {  
  lockBoard = true;  
  setTimeout(unflipCards, 800); // ここの数字だけを変える  
}
```

短くすると絵柄を覚える時間が減って難しくなり、長くすると次の手まで待たされてテンポが落ちます。`300` と `1500` を実際に試すと、遊びやすさがこの数字ひとつで決まることが分かります。

このあいだは `lockBoard` が `true` なので、クリックを一切受け付けません。長くするほど「押しても反応しない時間」も伸びる、という副作用込みで選びます。

応用課題 2: 絵柄を変える

`symbols` の中身を書き換えるだけです。絵文字でも文字でも構いません。

```
const symbols = ["🍎", "🍌", "🍇", "🍓", "🍊", "🍈", "🍑", "🍍"];  
  
// 書き換えた例  
const symbols = ["犬", "猫", "鳥", "魚", "馬", "羊", "鼠", "兔"];
```

8 個のまま増減しなければ、他はどこも直さずに済みます。`deck` は `symbols` から作られ、一致判定は `dataset.symbol` の比較なので、中身が何であっても同じように動きます。

気をつけるのは 2 点です。同じものを 2 回書くと 4 枚が同じ絵柄になり、ペアの数え方が合わなくなります。長い文字を入れるとカードからはみ出すので、そのときは文字サイズを下げます (`examples/challenges/pair-rule.js` が `back.style.fontSize` でそうしています)。

応用課題 3: ペア数を変える

`symbols` の数を変えると、枚数はその 2 倍で変わります。列数は `styles.css` が持っているので、2 か所を合わせます。

```
// script.js - 10 ペア = 20 枚にする
const symbols = ["🍎", "🍌", "🍇", "🍓", "🍊", "🍓", "🍑", "🍍", "🍒", "🍋"];
```

```
/* styles.css - 20 枚を 5 列 4 行に並べる */
#board {
  grid-template-columns: repeat(5, 1fr);
}
```

列数が枚数を割り切らないと、最後の行だけ欠けた並びになります。20 枚なら 4 列か 5 列、12 枚 (6 ペア) なら 4 列のままで 3 行に収まります。

ペア数の表示とクリア判定はどちらも `symbols.length` から作っているので、こちらは直さなくても追従します。

応用課題 4: 記憶タイム

`resetGame` の `renderBoard()`; の下に数行足して、伏せに戻す処理を関数にすると、始まる前に全部のカードを 3 秒だけ見せられます。

```
// resetGame の renderBoard(); の下
const allCards = document.querySelectorAll(".card");
for (let i = 0; i < allCards.length; i++) {
  allCards[i].classList.add("flipped");
}
setTimeout(hidePreview, 3000);

// 他の関数と並べて置く
function hidePreview() {
  const cards = document.querySelectorAll(".card");
  for (let i = 0; i < cards.length; i++) {
    cards[i].classList.remove("flipped");
  }
}
```

使っているのは、めくる処理で書いた `classList` と、不一致のときに書いた `setTimeout` だけです。それでも、運任せだったゲームが記憶を試すゲームに変わります。

`3000` を `1000` にすると一気に難しくなります。ちょうどいい長さを探してみてください。

応用課題 5: ベストスコアを保存する

クリアしたときの手数を残して、次にページを開いたときも「これまでの最小手数」を出せるようにします。ページを閉じてでも消えない置き場所が必要です。

やること

- クリア判定のところで、今回の手数がこれまでの最小より少ないか調べる
- 少なければ保存して、クリアメッセージを「自己ベスト更新！」に変える
- 更新できなかったときは、これまでのベストを併記する

ヒント

- 使うのは `localStorage` です。 `setItem(キー, 値)` で書き、 `getItem(キー)` で読みます
- 保存できるのは文字列だけなので、大小を比べる前に数値へ戻す必要があります
- いちばん最初はまだ何も入っていません。読んだ結果が空のときの分岐を先に考えます

答えは付録にあります。

応用課題 6: リセットに残るバグを直す

完成したコードには 1 つバグが残っています。まず再現させてみてください。

再現手順

1. 違う絵柄の 2 枚をめくる
2. 伏せに戻る前に「もう一度」を押す
3. 新しい盤面で 1 枚めくり、そのカードをもう一度押す

同じカード 1 枚がペア成立と判定され、ペア数が 1 つ進みます。Console には `Cannot read properties of null` が出ています。

ヒント

- 手順 2 と 3 のあいだに、誰も押していないのに動いた処理があります
- 5-3 でタイマーを止めるときに使った仕組みが、ここにも要ります

原因と直し方は付録にあります。

応用課題 7: ペアの条件を変える

「同じ絵柄どうし」ではなく「英単語とその和訳」でペアにします。元素記号と元素名、都道府県と県庁所在地でも同じ作りです。

表に出す文字 (`label`) と、判定に使う印 (`pairId`) をカードに別々に持たせるのがコツです。

```
const PAIRS = [ ["dog", "犬"], ["cat", "猫"], ["bird", "鳥"], /* ... */ ];

// 組ごとに 2 枚作る。同じ組の 2 枚には同じ pairId を付ける
function buildDeck() {
  const cards = [];
  for (let pairId = 0; pairId < PAIRS.length; pairId++) {
    cards.push({ label: PAIRS[pairId][0], pairId: pairId });
    cards.push({ label: PAIRS[pairId][1], pairId: pairId });
  }
  return shuffle(cards);
}

const isMatch = firstCard.dataset.pairId === secondCard.dataset.pairId;
```

`deck` の中身が文字列からオブジェクトに変わるので、`createCard` が受け取るものも変わります。全文は `examples/challenges/pair-rule.js` です。ルールを変えたら、Share ボタンで人に遊んでももらえます。絵柄を変えたときより反応があります。

応用課題 8: 難易度切り替え (4×4 / 6×6 / 8×8)

難易度で変わるのは列数とペア数の 2 つです。バラバラに持たずに 1 つの表にまとめると、切り替えが数行で済みます。

```
const LEVELS = {
  easy: { label: "4×4", cols: 4, pairs: 8 },
  normal: { label: "6×6", cols: 6, pairs: 18 },
  hard: { label: "8×8", cols: 8, pairs: 32 },
};

function resetGame() {
  const level = LEVELS[currentLevel];
  symbols = ALL_SYMBOLS.slice(0, level.pairs);
  boardEl.style.gridTemplateColumns = `repeat(${level.cols}, 1fr)`;
  // 以下はそのまま
}
```

応用課題 3 では `styles.css` を手で書き換えましたが、遊びながら切り替えるなら JS 側から `style` を触ります。

`symbols` は難易度で中身が変わるので `const` ではなく `let` にします。

難易度ボタン自体も `createElement` で作れます。全文は `examples/challenges/difficulty.js` です。

応用課題 9: カウントダウンモード

60 秒でクリアできなければゲームオーバーにします。5-3 で作ったタイマーはそのまま使えて、表示する数を経過から残りに変えるだけです。

```
const TIME_LIMIT = 60;
let isGameOver = false;

function renderTimer() {
  const elapsed = Math.floor((Date.now() - startTime) / 1000);
  const remaining = Math.max(0, TIME_LIMIT - elapsed);
  // remaining を mm:ss に整形して timerEl に入れる
  if (remaining === 0) gameOver();
}

function gameOver() {
  stopTimer();
  isGameOver = true;
  clearMessageEl.textContent = `時間切れ! ${matchedPairs} / ${symbols.length} ペア`;
}
```

盤面を止めるのに `lockBoard` は使えません。あれは 1 ターンごとに `resetTurn` が `false` に戻るためです。時間切れは別の状態として `isGameOver` で持ち、`handleCardClick` の先頭で見ます。全文は `examples/challenges/countdown.js` です。

応用課題 10: 状態を state にまとめる

今日は「状態を +1」と「画面に反映」の 2 行 1 セットを何度も書きました。片方を書き忘れると画面だけ古いままになります。状態を変える入口を 1 つに絞ると、この 2 行が 1 行になります。

```
const state = { moves: 0, matchedPairs: 0, /* ... */ };

// 状態を変える唯一の入口。変えたら必ず描画も走る
function setState(patch) {
  Object.assign(state, patch);
  render();
}

// state を読むだけで、state は変えない
function render() {
  movesEl.textContent = state.moves;
  pairsEl.textContent = `${state.matchedPairs} / ${symbols.length}`;
}

setState({ secondCard: card, moves: state.moves + 1 }); // 2 行が 1 行になる
```

めくれているかどうかだけは `state` に入れず、カードの `flipped` クラスのままにしています。ここも `state` に寄せると、めくるたびに盤面を作り直すことになり、CSS のアニメーションが再生されないためです。全文は `examples/challenges/state.js` です。

付録: 今日出てきた道具

ブラウザが読み込んだ HTML の構造を DOM と呼びます。

今日 JavaScript から要素を作ったり書き換えたりしていたのは、DOM 操作と呼ばれる作業です。

- 動的 DOM 生成 — `createElement()`, `appendChild()`, `replaceChildren()`
- DOM とデータの紐付け — `dataset`
- 中身の文字列の置き換え — `textContent`
- クラス操作 — `className`, `classList.add()`, `remove()`, `contains()`
- イベント — `addEventListener("click", ...)`
- 非同期 — `setTimeout()`, `setInterval()`, `clearInterval()`
- アルゴリズム — Fisher-Yates シャッフル
- 文字列整形 — テンプレートリテラル、`padStart()`
- 設計 — 状態 → 描画の分離

付録: 応用課題 5 の答え — ベストスコア

`localStorage` はブラウザにデータを残す仕組みです。ページを閉じてでも消えません。

```
// 最小手数を更新できたら保存する。更新したときだけ true を返す
function saveBest(currentMoves) {
  const best = Number(localStorage.getItem("bestMoves"));
  if (!best || currentMoves < best) {
    localStorage.setItem("bestMoves", currentMoves);
    return true;
  }
  return false;
}
```

`handleMatch` のクリア判定のところで呼びます。

```
if (matchedPairs === symbols.length) {
  stopTimer();
  if (saveBest(moves)) {
    clearMessageEl.textContent = `自己ベスト更新! ${moves}手 / ${timerEl.textContent}`;
  } else {
    clearMessageEl.textContent = `クリア! ${moves}手 (ベスト ${localStorage.getItem("bestMoves")}手)`;
  }
}
```

保存できるのは文字列だけなので、数値として比べるときは `Number(...)` で戻します。

付録: 応用課題 6 の答え — リセットに残るバグ

原因は 3-3 の `setTimeout(unflipCards, 800)` です。リセットで盤面は作り直されますが、ブラウザに預けた予約はそのまま残り、800ms 後に発火します。発火した `unflipCards` は、リセットで `null` に戻った `firstCard` を読もうとして落ちます。

`setTimeout` の返り値を控えておいて、リセットのときに取り消します。

```
let unflipTimerId = null; // 他の状態変数と並べて置く

function handleMismatch() {
  lockBoard = true;
  unflipTimerId = setTimeout(unflipCards, 800); // id を控えておく
}

function resetGame() {
  stopTimer();
  clearTimeout(unflipTimerId); // 予約も取り消す
  // 以下はそのまま
}
```

5-3 で `setInterval` に `clearInterval` を用意したのと同じ形です。ブラウザに預けたものは、要らなくなったら取り消します。

付録: sort でシャッフル — ダメな例

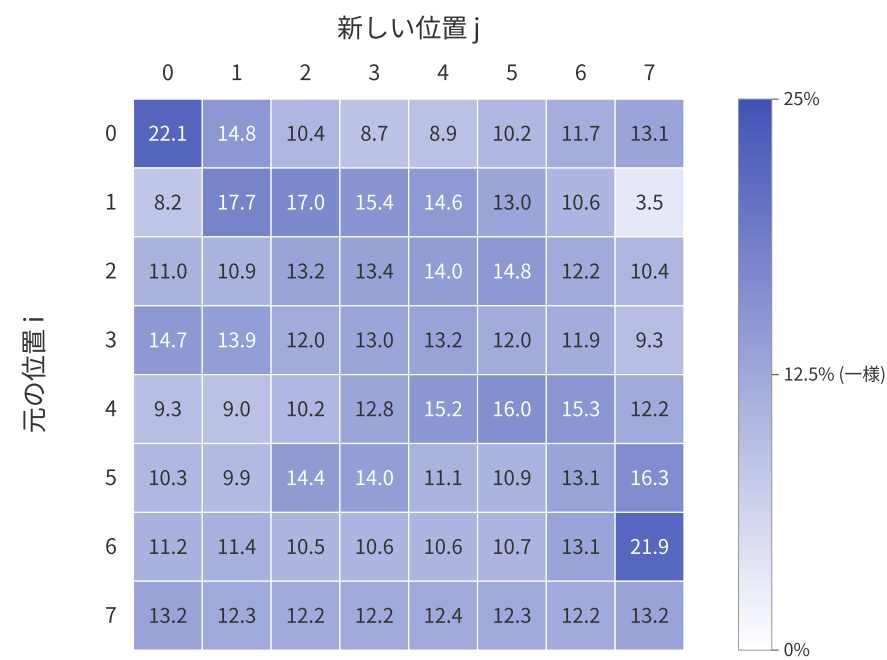
「配列を JavaScript でシャッフル」と検索すると、次の 1 行がよく紹介されています。

```
// これはダメな例です  
deck.sort(() => Math.random() - 0.5);
```

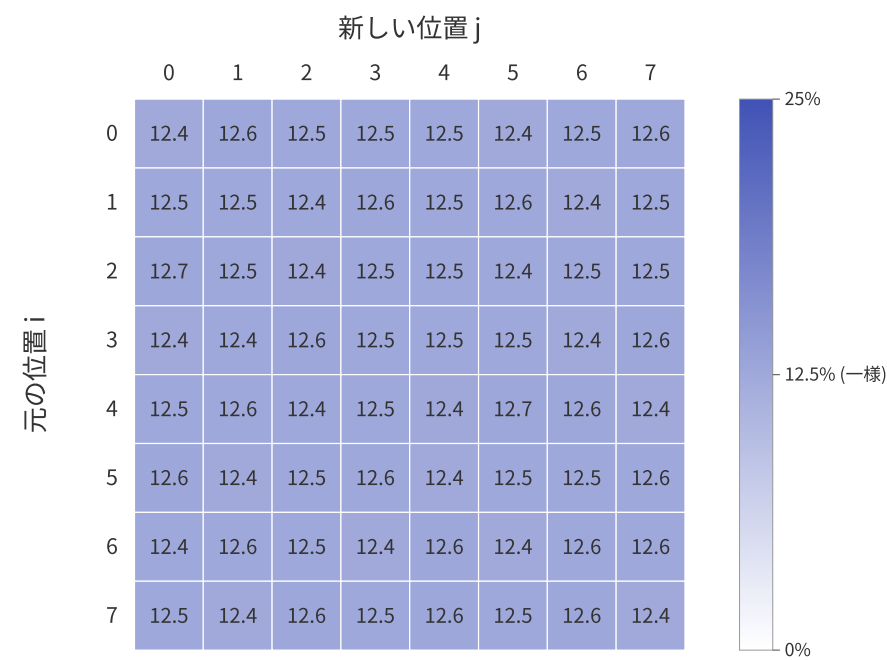
短くて動くように見えるので広まっていますが、`sort` はもともと「小さい順に並べる」ための関数で、シャッフル用ではありません。無理に使うと特定の位置に偏りが出ます。

付録: sort でシャッフル — 分布を比べてみる

同じ条件 (N=8, 200,000 回) で `sort` と Fisher-Yates を回した結果です。左は特定のセルが濃くなっているのに対し、右は全セルが 12.5% 前後に収まります。



N=8, 200,000 回試行
一様なら全セル 12.5% になるはずが、左上・右下に濃い部分が残る



N=8, 200,000 回試行
全セルがほぼ 12.5% で均等

付録: sort でシャッフル — なぜ偏るのか

要素が 3 つのときで考えます。

理想: 並び方は $3! = 6$ 通り。一様シャッフルなら、6 通りが各 $1/6 \approx 16.7\%$ で出るべきです。

実際: `sort(() => Math.random() - 0.5)` の内部で作れる「比較の答えの組み合わせ」は、比較を 3 回行う実装なら全部で 27 通り ($= 3^3$) あります。27 通りの結果を 6 通りの並びに割り当てると、 $27 \div 6 = 4$ あまり 3。あまりのぶんが偏りとして現れます。

並び	出現確率
132, 213, 231	$5/27 \approx 18.5\%$
123, 312, 321	$4/27 \approx 14.8\%$

比較の回数は処理系によって変わるため、上の数値も実装依存です。ただし n^n が $n!$ の倍数にならない限り、要素数を増やしても偏り自体は残ります。

詳しくは → [シャッフルした結果が偏ると相談を受けたときに確認すること \(Zenn\)](#)

付録: const と let — 使い分け

`const` は再代入できない宣言、`let` は再代入できる宣言です。

```
const a = 1;
a = 2;      // TypeError: Assignment to constant variable.

let b = 1;
b = 2;      // これは通る
```

書き換えないとわかっているものを `const` にしておくと、うっかり別の値を入れてしまったときにその場でエラーになります。先に `const` で書いて、書き換えが必要になった時点で `let` に直すのが楽です。

今日のコードでは `boardEl` のような要素の参照や `createCard` の中の一時変数が `const`、`firstCard` や `moves` のように遊んでいる間に変わるものが `let` です。

付録: const と let — 中身の書き換え

`const` が禁じているのは変数への再代入だけです。配列やオブジェクトの中身を書き換えるのは `const` でも通ります。

```
const arr = [1, 2, 3];  
arr[0] = 9;    // 通る。arr が指している配列は同じまま  
arr.push(4);   // 通る  
arr = [9, 9];  // TypeError。別の配列に差し替えるのはダメ
```

4-1 の `shuffle` が `const result` の中身を入れ替えられたのはこのためです。一方 4-2 で `deck` を `let` に変えたのは、`shuffle` が返す別の配列を入れ直すからで、こちらは再代入にあたります。

付録: `===` と `==`

`===` は値と型を両方チェックする厳密な比較演算子です。3-1 の一致判定で使いました。

- `"🍎" === "🍎" → true`、`"🍎" === "🍇" → false` (いまの一致判定)
- `"1" === 1 → false` (`"1"` は文字列、`1` は数値で型が違う)
- `"1" == 1 → true` (`==` は型を揃えてから比べるので通ってしまう)

`==` は比べる前に型を揃えるので、揃えた結果たまたま一致したものまで `true` になります。特別な理由がない限り `===` を使うのが定石です。

付録: dataset は HTML でどう見えるか

`card.dataset.symbol = "🍎"` を実行すると、ブラウザが持っている HTML の上では次のように追加されます。

```
<!-- 書き足すものではない -->  
<div class="card" data-symbol="🍎">
```

`dataset` の後ろに置いた名前が、`data-` を付けた属性名になります。`dataset.symbol` なら `data-symbol`、`dataset.color` なら `data-color` です。

付録: アロー関数

`=>` を使った短い関数の書き方です。2-2 でカードをクリックを繋ぐときに使いました。

```
// 名前を付けた関数
function add(a, b) {
  return a + b;
}

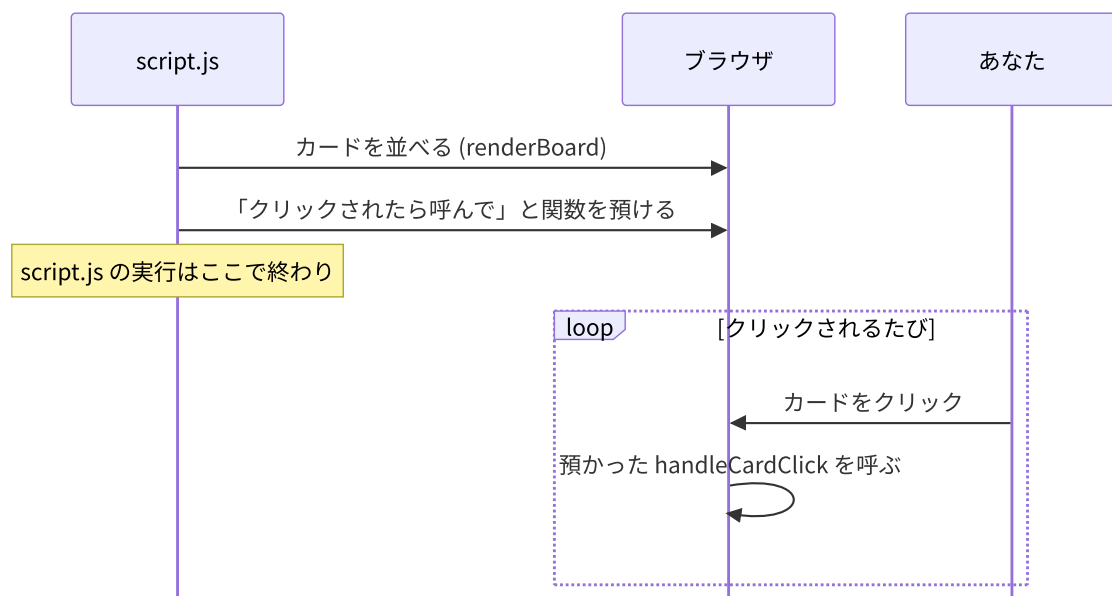
// 同じものをアロー関数で書く
const add = (a, b) => a + b;
```

`=>` の左が引数、右が実行する処理です。処理が 1 つの式だけなら、`return` も波カッコも省けます。

2-2 の `() => handleClick(card)` は、`=>` の左が空で、右で `card` を渡して `handleClick` を呼んでいます。この処理を作って預けておくことで、クリックされたときに初めて中の呼び出しが動きます。

付録: ブラウザが関数を呼ぶ仕組み (イベント駆動)

`script.js` が実行するのは、変数の宣言と `renderBoard()` の呼び出しだけです。それでもクリックするとカードがめくれます。 `handleCardClick` を呼んでいるのは誰でしょうか。



ブラウザです。 `addEventListener` で関数を預けると、待つのも呼ぶのもブラウザがやります。この作りをイベント駆動と呼びます。

不一致のカードを伏せに戻す `setTimeout` も同じ形です。きっかけがクリックから時間になるだけです。

付録: setTimeout と setInterval の使い分け

関数	いつ使う	止め方
<code>setTimeout(fn, ms)</code>	一度だけ、少し待ってから実行 (3-3 のミスマッチ)	<code>clearTimeout(id)</code>
<code>setInterval(fn, ms)</code>	一定間隔で繰り返し実行 (5-3 のタイマー)	<code>clearInterval(id)</code>

どちらも「関数をブラウザに預けて、あとから呼んでもらう」形は同じです。違うのは 1 回きりか、繰り返しかという点だけです。

`setInterval` は `clearInterval` を呼ぶまで止まりません。不要になったら必ず止めます。

付録: なぜ HTML の `onclick=""` ではないのか

- `element.addEventListener("click", fn)` — JavaScript 側で管理したい (基本これ)

- `<div onclick="...">` — 手軽な確認用のプロトタイプ

`addEventListener` を使う理由:

- HTML と JS の役割を分けられる (責務の分離)
- 同じイベントに複数の関数を付けられる
- 削除も柔軟にできる

```
<script> document.querySelectorAll('.timer-box').forEach(box => { const el = box.querySelector('.timer'); const
initial = Number(box.dataset.seconds); let remain = initial; let id = null; const render = () => { const r =
Math.max(remain, 0); const m = String(Math.floor(r / 60)).padStart(2, '0'); const s = String(r % 60).padStart(2,
'0'); el.textContent = `${m}:${s}`; el.classList.toggle('warn', remain <= 60 && remain > 0);
el.classList.toggle('done', remain <= 0); }; const stop = () => { clearInterval(id); id = null;
el.classList.remove('running'); }; const start = () => { if (remain <= 0) return; el.classList.add('running'); id =
setInterval(() => { remain--; render(); if (remain <= 0) stop(); }, 1000); }; el.addEventListener('click', () => { if
(remain <= 0) { el.classList.remove('done'); return; } id ? stop() : start(); }); el.addEventListener('contextmenu', e
=> { e.preventDefault(); stop(); remain = initial; render(); }); box.querySelectorAll('.timer-btn').forEach(btn => {
btn.addEventListener('click', () => { remain = Math.max(0, remain + Number(btn.dataset.delta)); render(); }); });
render(); }); </script>
```